Введение в NLP

NLP tasks, preprocessing, text representations, classical classifiers и RNN

Алан-Барсаг Газзаев

24 октября 2026 10:10–11:40

Содержание первой лекции

Задачи, данные и preprocessing

Основные задачи NLP различаются формой ответа: class label, последовательность меток, найденный документ или новый текст. После краткой истории подходов рассматриваются источники данных, annotation и разделение corpus. Затем — tokenization, normalization, lemmatization, stemming и stop words, с примерами того, как эти операции меняют доступную модели информацию.

Text representations, classifiers и RNN

Zipf’s law, n-grams, Bag of Words и TF-IDF вводятся как базовые понятия. Naive Bayes и Logistic Regression рассматриваются на уровне принципа работы и применения. Раздел RNN объясняет обработку последовательности без разбора внутренней архитектуры. Завершают лекцию библиотеки; реализация preprocessing и classification переносится на семинар.

Natural Language Processing

Предмет NLP

Natural Language Processing объединяет вычислительные методы работы с человеческим языком. Входом может быть сообщение, документ, диалог или пара текстов. Постановка задачи задаёт доступный контекст, единицу анализа и формат ответа. Одна и та же модель не обязана одинаково хорошо решать задачи с разными требованиями к результату.

Источники сложности

Язык содержит неоднозначность: смысл слова зависит от контекста, а одинаковое содержание выражается разными формулировками. Отрицание и порядок слов меняют интерпретацию; опечатки и сокращения меняют наблюдаемую запись. Качество NLP-системы определяется не только алгоритмом, но и данными, правилами annotation и соответствием evaluation реальному применению.

Основные задачи NLP

Постановки различаются единицей анализа и формой ответа. Это различие определяет состав разметки, требования к модели и способ оценки результата.

Прикладной сценарий	Тип задачи
Распределение обращений по отделам	Text classification
Выделение адреса из сообщения	Information extraction
Подбор инструкции по запросу	Information retrieval
Сокращение переписки	Summarization

Например, отзыв можно отнести к классу тональности, выделить в нём название товара или найти по запросу. Для каждой операции требуется собственный целевой ответ и критерий правильности.

Text classification

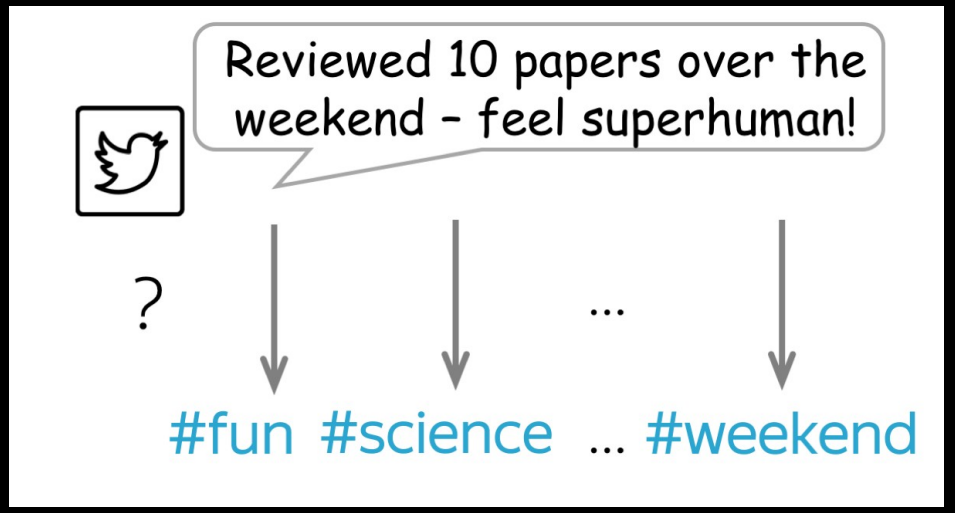
Постановка задачи

Text classification сопоставляет документу class label из заранее заданного набора. Документом может быть короткий запрос, отзыв, письмо или целая статья. Примеры — spam detection, topic classification, intent classification и sentiment analysis. До обучения необходимо определить, какое свойство текста должна описывать метка и какой контекст доступен при предсказании.

Объект, представление и ответ

Обозначим исходный документ через x, его feature representation через h, а целевую метку через y. Feature extractor преобразует текст в численный вектор; classifier использует этот вектор для выбора класса. В single-label постановке каждому документу соответствует одна правильная метка. Для multi-label задачи набор допустимых ответов устроен иначе.

Single-label и multi-label classification



В single-label classification у объекта одна целевая метка: binary classification задаёт два возможных класса, multi-class — больше двух. В multi-label classification несколько меток могут быть верны одновременно. Например, сообщение может относиться и к доставке, и к оплате. Число классов и число меток у одного объекта — разные характеристики задачи. Это различие влияет на annotation, выход модели и выбор evaluation metrics.

Sentiment analysis: постановка задачи

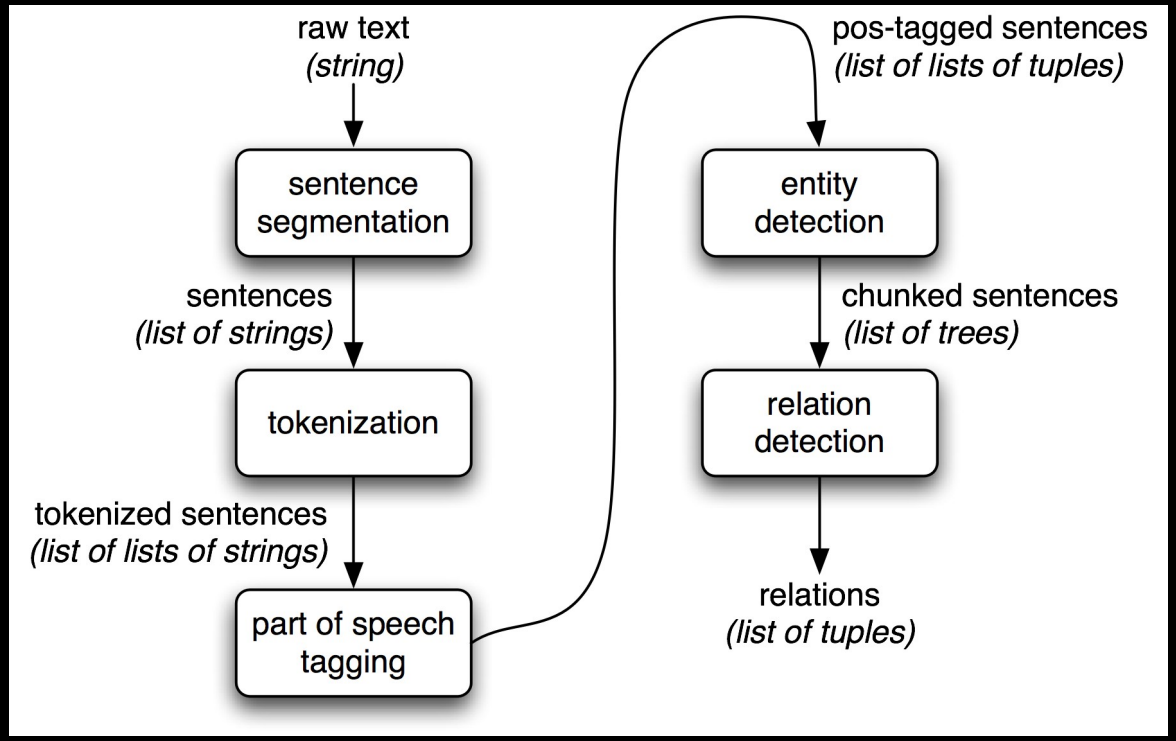
Объект оценки

Отзыв «Товар отличный, но доставка заняла две недели» содержит оценки разных объектов. Положительное отношение к товару сочетается с недовольством доставкой. Прежде чем назначать метку, необходимо определить, что именно оценивает система: товар, обслуживание, заказ в целом или отдельные аспекты пользовательского опыта.

Правила интерпретации

Одинаковый текст может получать разные допустимые метки в разных постановках. Инструкция должна объяснять, как учитывать смешанные оценки, отсутствие явного мнения, иронию и нейтральные факты. Неоднозначность исходной задачи ограничивает качество модели: дополнительное обучение не заменяет согласованных определений классов и правил разметки.

Information extraction



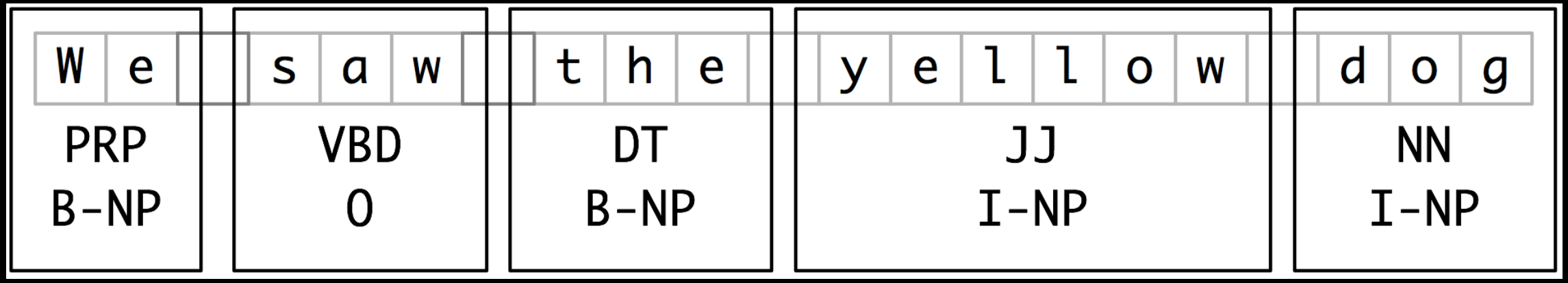
Пример последовательного извлечения сущностей и отношений.

Источник: NLTK Book, гл. 7

Схема показывает один из вариантов извлечения информации: выделение предложений и токенов, определение частей речи, поиск сущностей и установление отношений.

Сущность должна иметь правильные границы и категорию. Отношение связывает найденные сущности, например организацию и место её расположения. Ошибка раннего этапа может повлиять на последующие результаты.

Sequence labeling: IOB



Метка B обозначает начало фрагмента, I обозначает его продолжение, O обозначает токен вне выделенных фрагментов. В примере NP соответствует именной группе. Аналогичный принцип используют для разметки сущностей, добавляя тип фрагмента к каждой метке. Качество зависит от правильности границ и категорий, поэтому ошибочная метка одного токена может изменить интерпретацию всего фрагмента.

Пример IOB-разметки из NLTK Book. Именная группа и именованная сущность являются разными понятиями.

Источник: NLTK Book, гл. 7

Information retrieval

Запрос и коллекция документов

Поисковая система получает запрос и выбирает из коллекции документы, которые могут удовлетворить информационную потребность пользователя. Запрос и документ часто различаются по длине и формулировке. Для поиска используют лексическое совпадение, числовые представления текста и дополнительные сигналы, связанные с содержанием или условиями поиска.

Отбор и ранжирование

На первом этапе можно быстро сформировать множество кандидатов, затем упорядочить их по ожидаемой релевантности. Пользователь видит ограниченную часть выдачи, поэтому положение подходящего документа имеет значение. Для проверки нужны запросы, коллекция и оценки релевантности. Оценка отдельного текста без учёта запроса не описывает качество поисковой системы.

Text generation и summarization

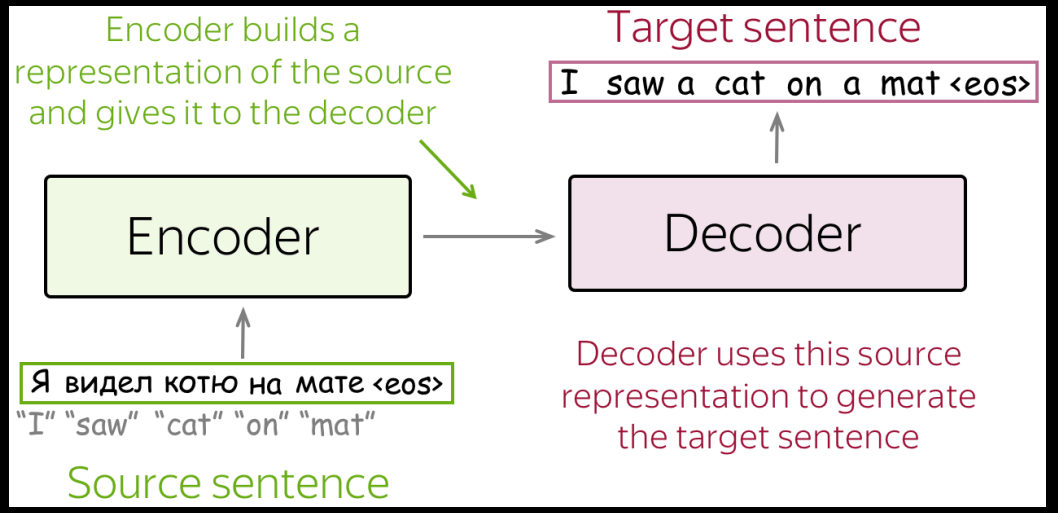
Формирование текста

Text generation создаёт последовательность tokens. Продолжение текста зависит от предыдущего контекста; ответ в диалоге дополнительно зависит от запроса и истории. В conditional generation исходные данные задают содержание результата, но обычно допускают несколько корректных формулировок. Поэтому точное совпадение с единственным эталоном не исчерпывает evaluation.

Summarization

Summarization сокращает исходный материал, сохраняя существенную информацию. Extractive подход выбирает фрагменты источника, abstractive допускает новые формулировки. Краткость сама по себе не означает качество: важны полнота значимых фактов, связность и отсутствие неподтверждённых утверждений. Language model может создавать грамматичный текст, который не соответствует исходным данным.

Machine translation



Machine translation получает текст на source language и формирует текст на target language. Требуется передать содержание, а порядок слов, длина и способы выражения могут измениться. Encoder–decoder — общий способ организовать conditional generation: encoder строит representation входа, decoder использует его при формировании ответа. Подробный разбор этой архитектуры относится к следующему блоку.

Rule-based и statistical NLP

Rule-based NLP

Ранние прикладные системы широко использовали словари, грамматики и правила, заданные экспертами. Правило может проверять наличие слова, шаблон записи или синтаксическую конструкцию. Такое поведение удобно контролировать в узкой области, но новые формулировки и исключения требуют ручной поддержки. Regex и словари по-прежнему полезны как отдельные компоненты обработки данных.

Statistical NLP

Statistical approaches оценивают закономерности по corpus: частоты слов, совместные появления, параметры classification или language modeling. Важными становятся размер и состав данных, feature engineering и независимая оценка. N-gram language models, Naive Bayes и линейные classifiers относятся к этому этапу. Переход между подходами происходил постепенно; они продолжают использоваться совместно.

Neural NLP и Large Language Models

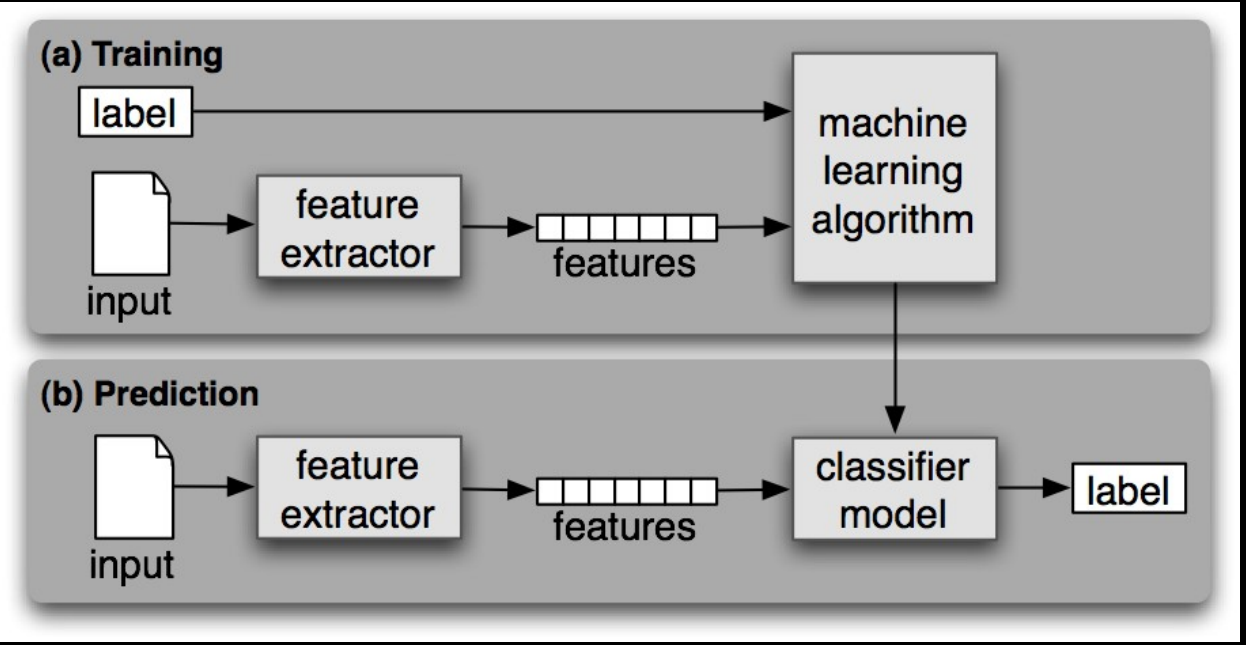
Neural NLP

Neural networks позволяют обучать feature representation по данным. Word embeddings задают численное описание слов, RNN и CNN обрабатывают последовательность и её локальные закономерности. С pretraining часть знаний переносится между задачами: модель сначала обучается на большом corpus, затем адаптируется к конкретному применению. Это уменьшает объём работы, который начинается с нуля.

Transformer и LLM

Transformer использует attention для взаимодействия между позициями последовательности. Масштабирование моделей и данных, pretraining и последующая адаптация привели к широкому применению LLM. Такие модели решают задачи через prompting и fine-tuning, но по-прежнему требуют качественных данных и evaluation. Первая лекция вводит необходимые понятия; архитектуры и способы применения разбираются в следующих блоках.

Supervised learning



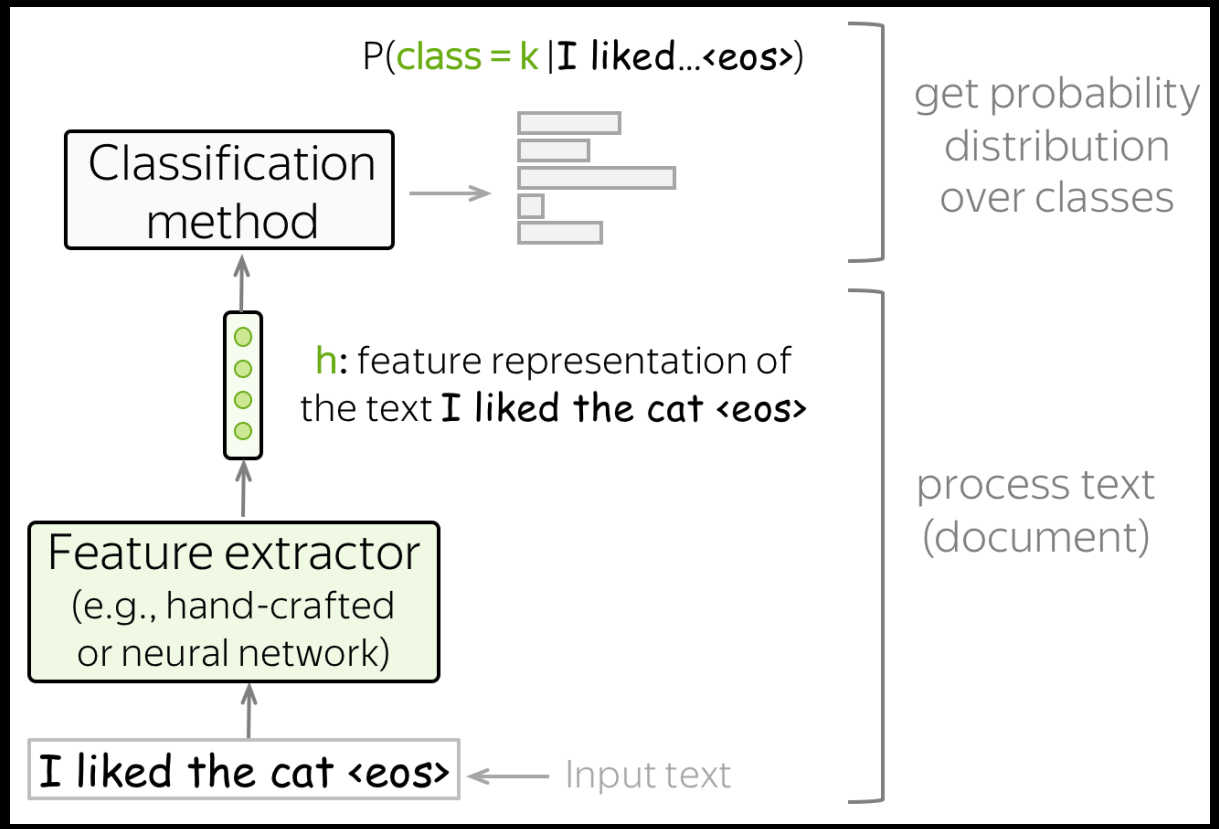
При обучении доступны входной текст и его целевая метка. Из текста получают признаки, а алгоритм оценивает параметры классификатора.

При применении модель получает признаки нового текста и предсказывает метку. Проверочные ответы используют только для оценки. Такая организация отделяет подбор параметров от проверки обобщения. Высокое качество на тренировочных примерах ещё не показывает, насколько успешно модель обработает новые документы.

Обучение использует размеченные примеры, применение использует сохранённую модель.

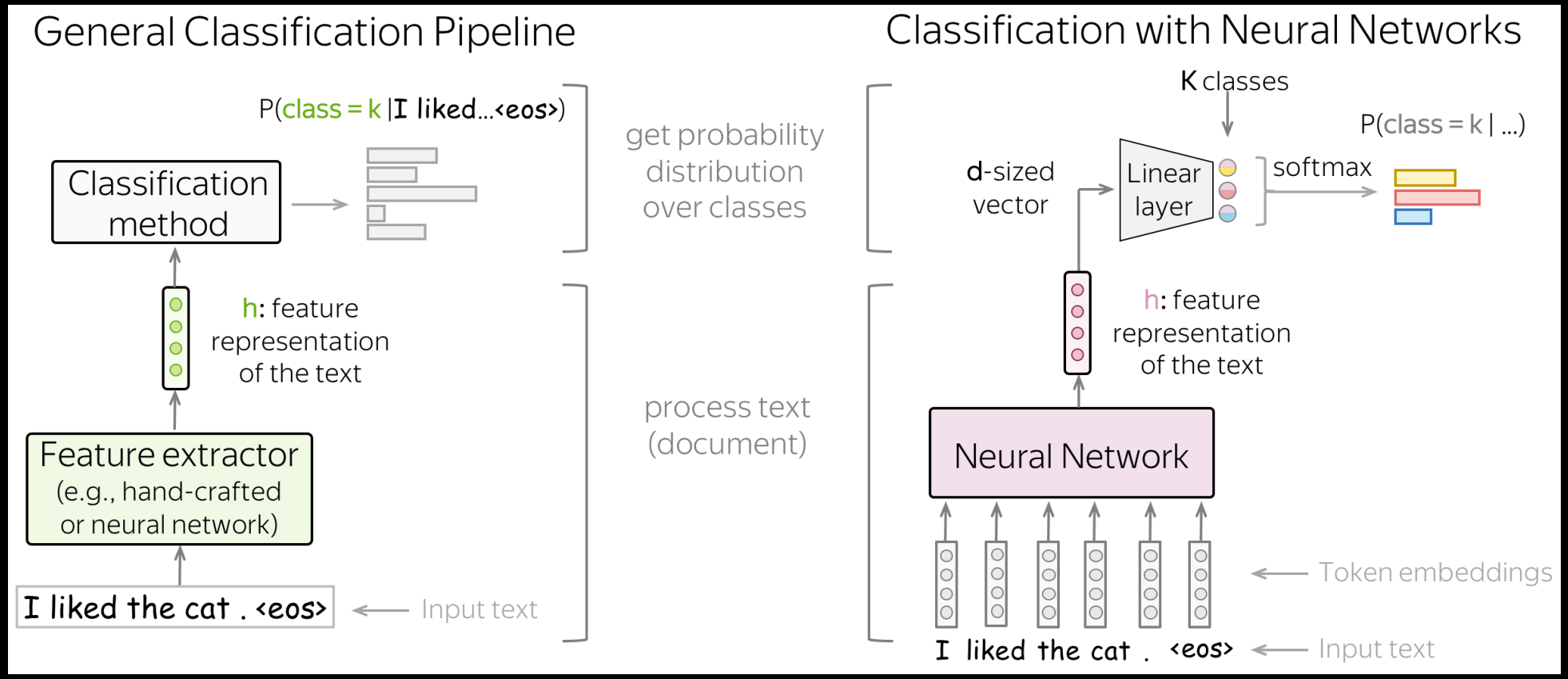
Источник: NLTK Book, гл. 6

General framework: features + classifier



Feature extractor получает исходный документ x и строит representation h. Classifier сопоставляет h оценкам классов и итоговому ответу. Разделение этих частей помогает планировать эксперимент: можно заменить TF-IDF на другое representation при сохранении classifier или сравнить classifiers на одинаковых features. Результат зависит от обеих частей и от данных, на которых оценивается качество.

Representation learning



Feature extractor строит representation текста. Classifier получает этот вектор и определяет class probabilities.

Text datasets: источники и состав

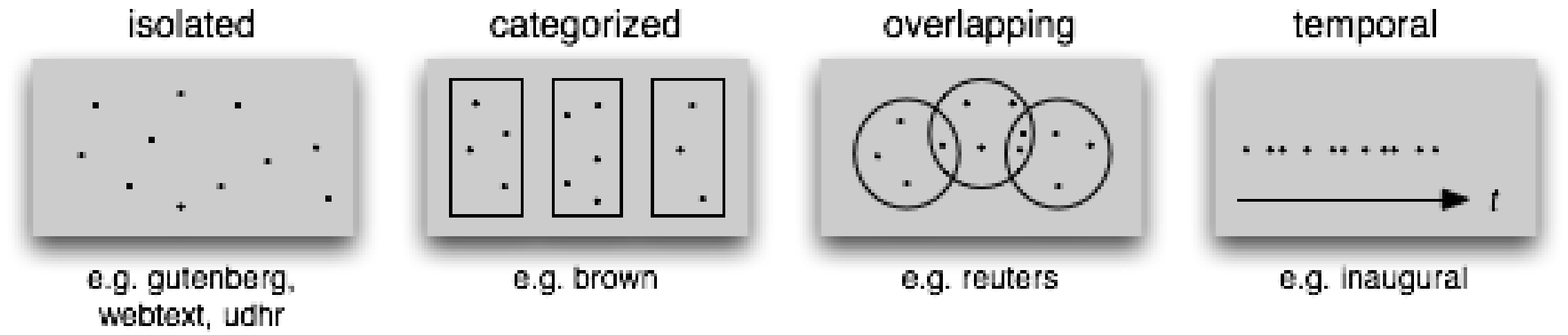
Соответствие будущему применению

Корпус может состоять из открытых наборов, продуктовых данных или специально собранных текстов. Доступность большого объёма не гарантирует соответствия задаче. Отзывы о фильмах отличаются от жалоб на доставку по лексике, типам оценок и структуре. При выборе данных учитывают язык, источник, период, жанр и правила получения целевых меток.

Проверка состава

До обучения анализируют пропуски, дубликаты, длины текстов и распределение классов. Полезно проверять данные по источникам и времени, чтобы обнаружить перекосы. Если отрицательные и положительные примеры поступили из разных каналов, модель может выучить особенности канала вместо тональности. Такой эффект необходимо отделять от решения исходной задачи.

Организация corpus



Документы можно хранить как независимую коллекцию, распределять по категориям, относить сразу к нескольким категориям или упорядочивать во времени. Структура корпуса определяет доступные сравнения и допустимые способы разбиения. Например, пересекающиеся категории соответствуют многометочной постановке, а временной порядок важен при проверке переноса модели на будущие данные.

Типы организации корпуса из NLTK Book: isolated, categorized, overlapping и temporal.

Источник: NLTK Book, гл. 2

Data collection

Выбор источника

Data collection начинается с определения будущего потока данных: кто пишет тексты, на каком языке, в каком жанре и с какой длиной. Источниками могут быть открытые corpora, разрешённые выгрузки из продукта, публичные документы или специально собранные примеры. Доступность коллекции ещё не означает, что она соответствует целевой задаче.

Состав и происхождение примеров

Для каждого документа полезно сохранять source, время, идентификатор автора или объекта и связь с другими документами. Эти поля помогают удалять дубликаты и строить group split или time-based split. Учитываются лицензии, разрешения на использование и наличие персональных данных. Ошибки encoding, пустые тексты и перекосы состава проверяют до annotation и обучения.

Annotation guidelines

Определение классов

Инструкция должна описывать единицу анализа, значения меток и правила выбора ответа. Для тональности необходимо определить объект оценки и обработку смешанных случаев. Рейтинг, поставленный пользователем, может служить источником метки, но его соответствие содержанию текста требует проверки. Пограничные случаи лучше пояснять конкретными примерами.

Согласованность разметки

Несколько разметчиков могут по-разному интерпретировать иронию, нейтральные факты и противоречивые оценки. Расхождения помогают обнаружить неясные определения классов и ошибки инструкции. После уточнения правил спорные примеры разбирают повторно. Согласованность меток важна для обучения и для того, чтобы ошибки модели имели однозначную интерпретацию.

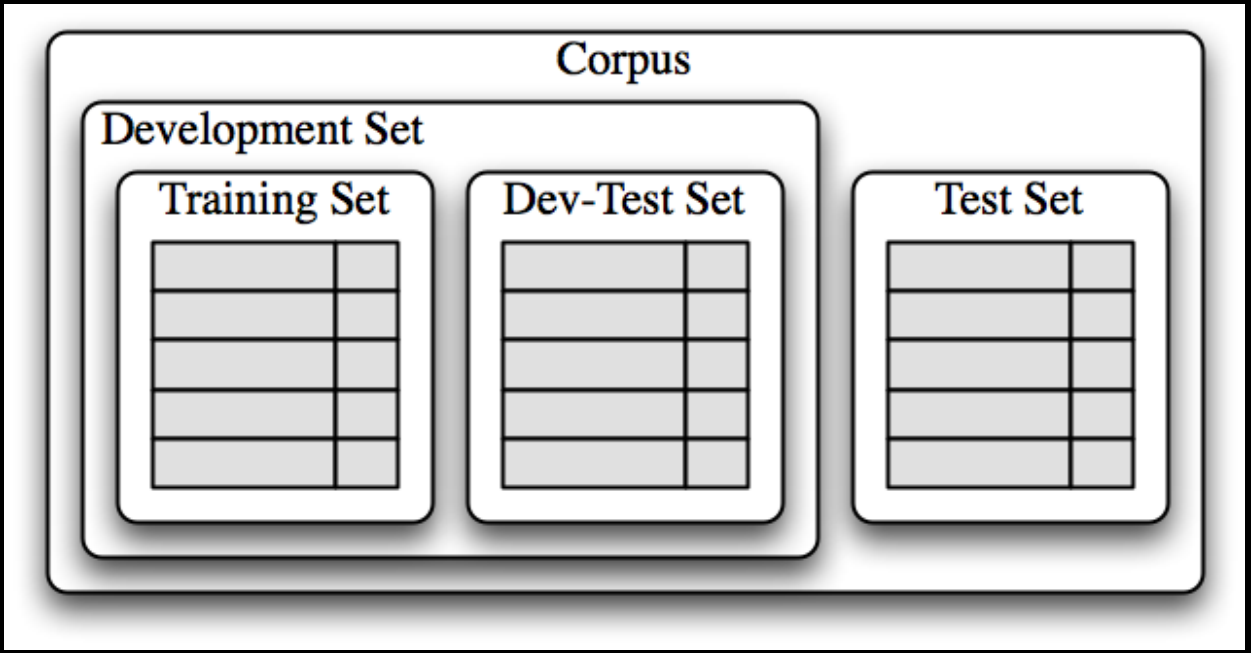
Datasets для text classification

Примеры из материалов курса ШАД. Dataset определяет задачу и правила работы с метками; готовая коллекция не заменяет анализ собственных данных.

Dataset	Задача	Особенность
SST-2 / SST-5	Sentiment analysis	Два или пять классов
IMDb Reviews	Sentiment analysis	Отзывы о фильмах
AG’s News	Topic classification	Четыре темы новостей
TREC	Question classification	Тип ожидаемого ответа

При выборе dataset проверяют язык, единицу annotation, label space, доступный контекст и официальное разбиение. Числа из разных версий коллекции могут различаться.

Train, dev и test



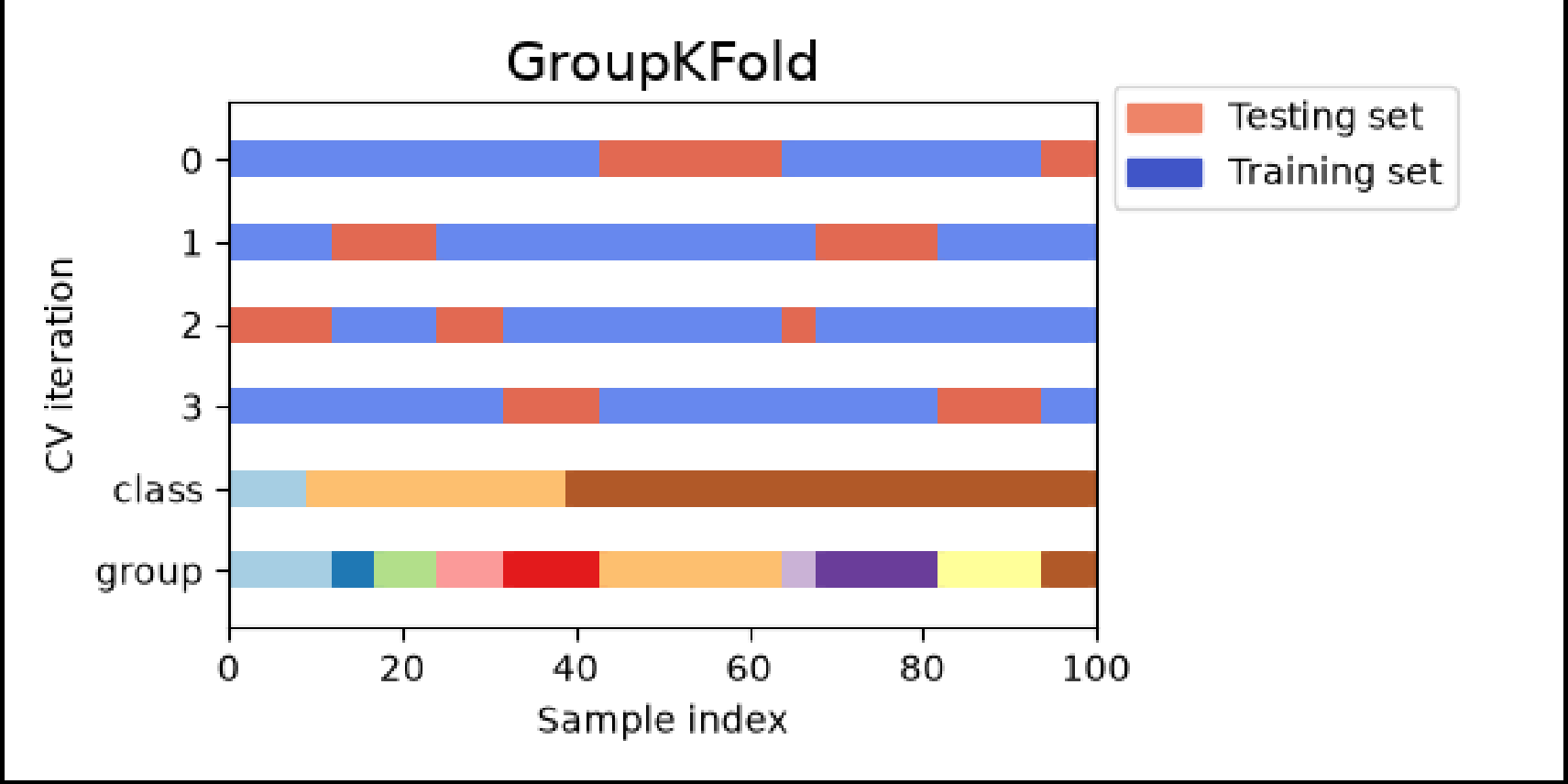
Training Set используют для обучения. Dev-Test Set в этой схеме соответствует валидации: по нему выбирают настройки и варианты решения. Test Set сохраняют для итоговой оценки.

Дубликаты и связанные документы не должны создавать утечку между частями. В зависимости от применения разбиение проводят по авторам, объектам или времени. Словарь и статистики признаков оценивают только по тренировочным данным внутри выбранной процедуры проверки.

Отдельные части корпуса предназначены для обучения, настройки и итоговой проверки.

Источник: NLTK Book, гл. 6

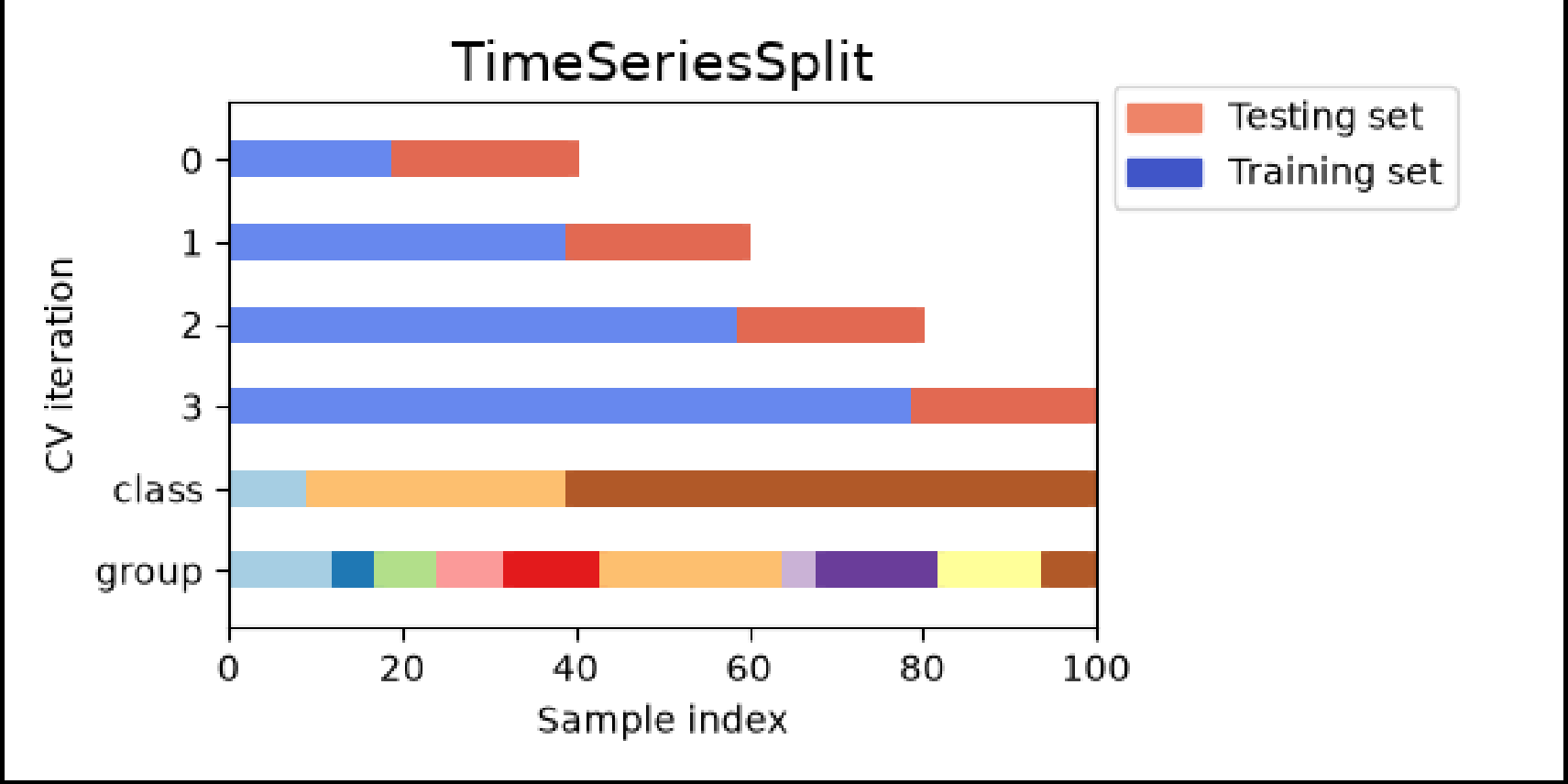
Group split



Синим показано обучение, оранжевым проверка. Документы одной группы не пересекают границу внутри разбиения.

Источник: Документация scikit-learn

Time-based split



Обучение использует прошлые данные, проверка использует следующий интервал. По оси X отложен порядок объектов.

Источник: Документация scikit-learn

Text preprocessing

Техническая согласованность

Preprocessing приводит исходные данные к виду, который допускает выбранный метод. Этот этап может включать обработку пробелов и разметки, выделение предложений и токенов, нормализацию записи и морфологические преобразования. Решения принимают с учётом источника данных: письмо, поисковый запрос и пользовательский отзыв имеют разные особенности.

Сохранение значимой информации

Любое преобразование меняет доступные модели сигналы. Регистр может указывать на название, пунктуация выражать эмоциональность, а отрицание определять смысл оценки. Поэтому максимальное сокращение текста не является самостоятельной целью. Сначала формулируют ожидаемую пользу операции, затем проверяют примеры и влияние на качество решения.

Этапы подготовки текста

На каждом этапе меняется форма представления данных. Техническая очистка и языковой анализ решают разные задачи.

Уровень	Пример результата	Назначение
Документ	Текст и исходная разметка	Сохранение источника
Извлечённый текст	Содержимое без навигации сайта	Выбор нужного материала
Токены	Отдельные элементы текста	Определение единиц анализа
Признаки	Числа в заданном пространстве	Вход классификатора

Перед удалением разметки и служебных частей проверяют, не содержат ли они нужные факты. Исходный документ полезно сохранять для проверки ошибок.

Token, type и vocabulary

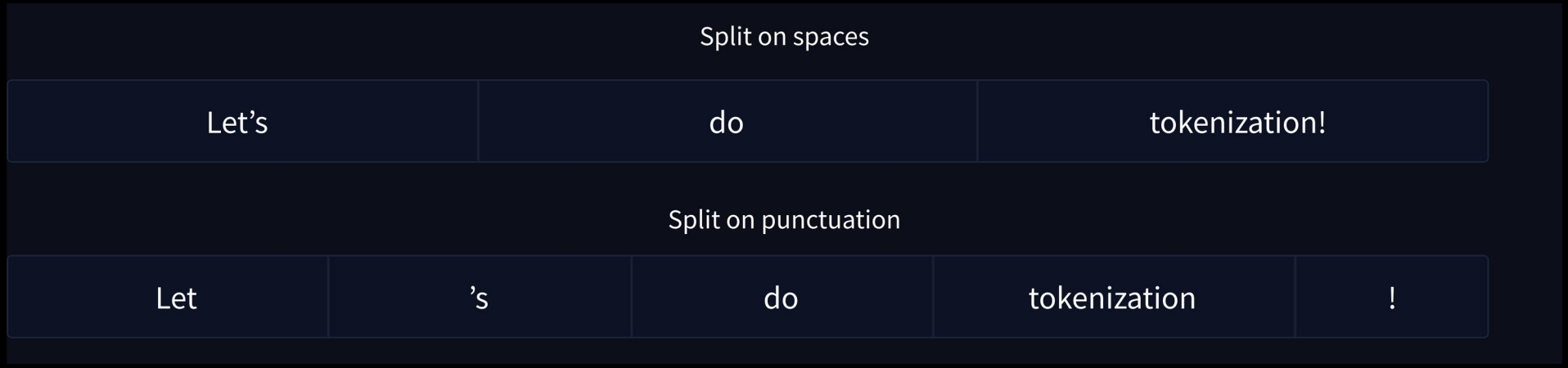
Token и type

Token — отдельное вхождение элемента в последовательность после tokenization. Type — уникальный элемент выбранного представления. В строке «кот видит кота» при word tokenization получается три tokens и три types. После lemmatization две словоформы могут получить общую lemma. Число вхождений и число различных элементов описывают разные свойства corpus.

Vocabulary и OOV

Vocabulary фиксирует набор допустимых types и их идентификаторы. Для Bag of Words он задаёт координаты вектора; для pretrained neural model — соответствие между token IDs и embeddings. OOV означает элемент вне vocabulary. Его обработка зависит от representation: игнорирование неизвестного слова, специальный token или разбиение на известные subwords.

Tokenization



Пример word tokenization англоязычного предложения.

На рисунке сравниваются разбиение по пробелам и учёт пунктуации. Во втором варианте восклицательный знак становится отдельным элементом, а сокращённая форма слова получает собственные границы. Tokenization определяет состав словаря и n-grams. Для русского текста отдельно проверяют дефисы, сокращения, числа и даты. На обучении и при применении модели необходимо сохранять одинаковые правила сегментации.

Источник: Hugging Face Course, гл. 2

Tokenization: rules и exceptions

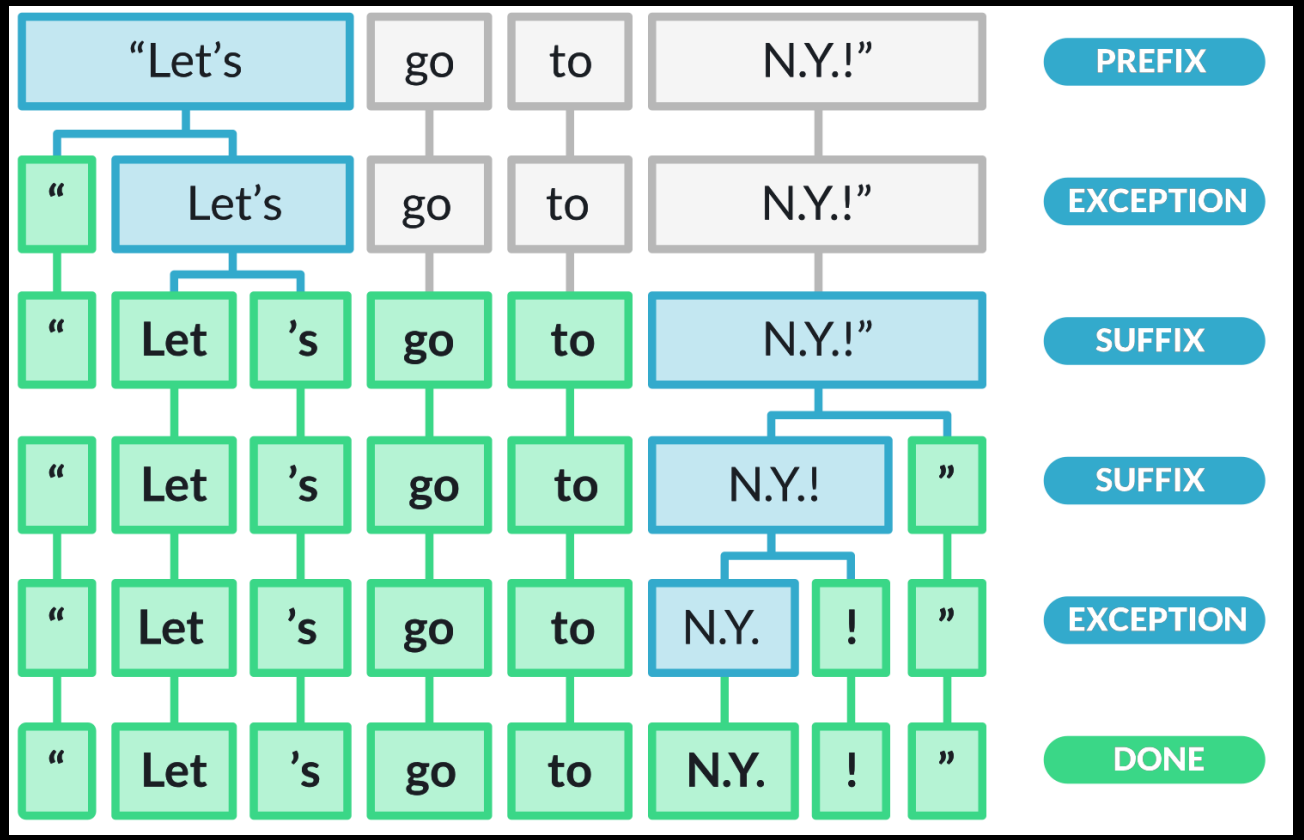


Схема из документации spaCy. Пример относится к английскому языку.

Источник: Документация spaCy

spaCy последовательно рассматривает начальные и конечные символы, а также известные исключения. На рисунке отдельно выделяются кавычки и восклицательный знак. Сокращение N.Y. сохраняет внутренние точки.

Простого разделения по пробелам недостаточно. Правила должны согласованно обрабатывать сокращения, пунктуацию и устойчивые формы записи.

Пограничные случаи tokenization

Правильное разбиение зависит от того, какая информация должна сохраняться в последующей задаче.

Запись	Существенное свойство	Что требуется проверить
Санкт-Петербург	Название с дефисом	Границы названия
24.10.2026	Единая запись даты	Связь частей даты
3,5 кг	Число и единица измерения	Десятичный разделитель
не понравился	Отрицание и оценка	Сохранение соседства

Таблица задаёт случаи для проверки. Она не предписывает одинаковое число токенов для всех библиотек и способов представления.

Обработка русского текста

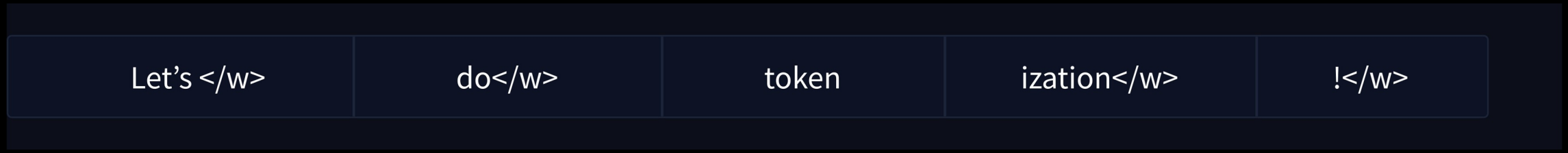
Словоформы и неоднозначность

В русском тексте одна lemma может встречаться во многих формах. Это увеличивает vocabulary при word-level representation. Одновременно одинаковая запись может допускать разные разборы: «стали» соотносится со словом «стать» или «сталь» в зависимости от контекста. Morphological analysis должен учитывать, какие свойства нужны конкретной задаче.

Запись и жанр

Проверяют буквы «е» и «ё», дефисы, сокращения, имена, числительные и смешение кириллицы с латиницей. Сообщения пользователей дополнительно содержат опечатки и нестандартные формы. Нельзя считать любой tokenizer или pretrained model одинаково пригодным для русского языка: важны language support и данные обучения. Проверка на характерных примерах дополняет общую метрику.

Word, character и subword tokenization



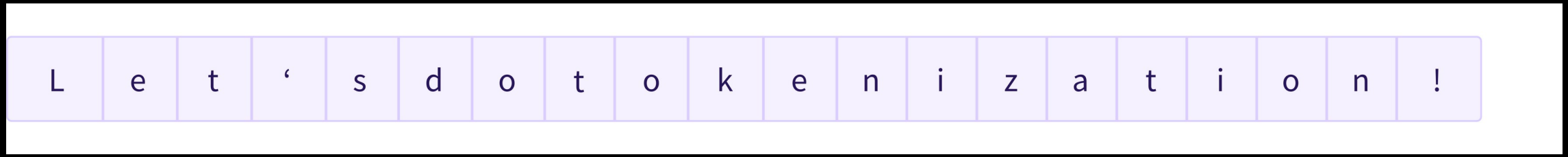
Subword units позволяют представить слово через несколько фрагментов.

Word tokenization использует целые слова, character tokenization использует отдельные символы, а subword выделяет промежуточные фрагменты. На рисунке слово tokenization разделено на token и ization. Маркер </w> обозначает конец слова в этом примере.

Subword units позволяют представить редкую запись через известные части. Они не обязаны совпадать с морфемами. Для предобученной модели применяют согласованный с ней tokenizer и словарь.

Источник: Hugging Face Course, гл. 2

Character-level representation

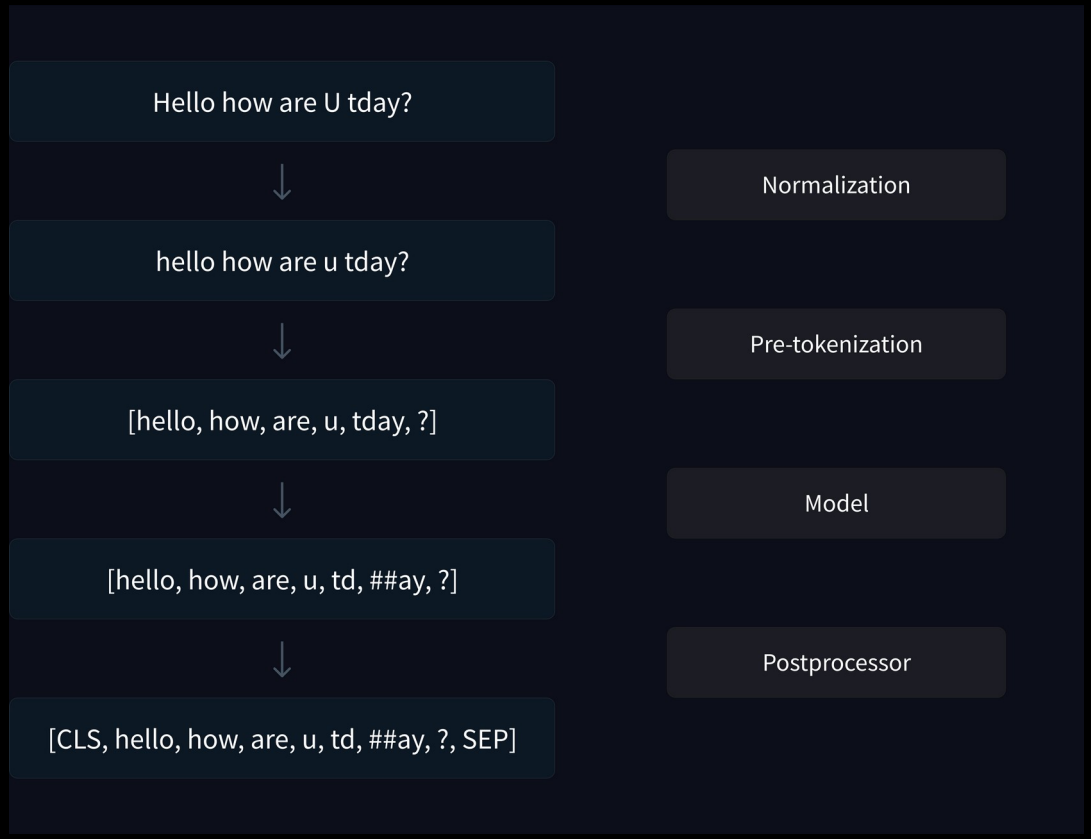


В символьном представлении слово разделяется на отдельные символы. Набор базовых единиц обычно меньше словаря целых слов, а новая словоформа может состоять из знакомых элементов. Последовательности становятся длиннее, и смысловые закономерности необходимо извлекать из сочетаний символов. В классической классификации часто используют character n-grams.

Иллюстрация из курса Hugging Face. Правила обработки пробелов зависят от конкретного tokenizer.

Источник: Hugging Face Course

Subword tokenization: общий pipeline



Сначала нормализация согласует запись текста. Предварительное разбиение выделяет крупные фрагменты. Модель tokenization делит их на единицы своего словаря, а заключительный этап может добавлять специальные маркеры.

Эти операции должны соответствовать требованиям выбранной языковой модели. Замена только одного этапа способна изменить весь набор входных идентификаторов.

Схема Hugging Face. Маркеры CLS и SEP показаны для конкретного семейства моделей и не являются универсальными.

Источник: Hugging Face Course

Text normalization

Normalization как выбор представления

Text normalization приводит варианты записи к выбранному соглашению: например, объединяет эквивалентные Unicode-последовательности или стандартизирует пробелы. Lowercasing уменьшает число различных форм, но может убирать различия между именами и обычными словами. Требуемые операции зависят от задачи и от того, какие сигналы нужно сохранить.

Согласованность обработки

Слишком агрессивная normalization уничтожает информацию: регистр, emoji, повторяющаяся пунктуация или числа могут влиять на class label. Для pretrained models применяют preprocessing, совместимый с их tokenizer. Решения фиксируют и одинаково используют на train, dev, test и новых документах. Пользу дополнительного преобразования проверяют по результатам evaluation.

Lemmatization и stemming

Оба подхода уменьшают разнообразие словоформ, но используют разные основания. Lemmatization опирается на языковой разбор, а stemming изменяет запись по формальным правилам.

Характеристика	Lemmatization	Stemming
Принцип	Определение словарной формы с учётом языковых признаков	Сокращение слова по набору правил
Пример	«машины» — «машина»	«машины» — «машин»
Результат	Лемма, обычно полноценное слово	Стем, который может не быть словом

Лемма обычно соответствует словарной форме. Стем может быть фрагментом, который не существует как самостоятельное слово. Приведение разных форм к одной записи полезно только при сохранении нужных различий.

Lemmatization: роль контекста

Одинаковая запись может соответствовать разным словарным единицам. Контекст помогает различить допустимые разборы.

Фрагмент	Словоформа	Интерпретация
Деталь изготовлена из стали	стали	Существительное «сталь»
Они стали работать быстрее	стали	Глагол «стать»
Новое стекло треснуло	стекло	Существительное «стекло»
Молоко стекло на пол	стекло	Глагол «стечь»

Примеры составлены для лекции. Lemmatization требует проверки неоднозначных форм, даже если отдельные распространённые слова обрабатываются правильно.

Morphological analysis

Lemma и grammatical features

Morphological analysis определяет возможную lemma, part-of-speech tag и grammatical features: например, число, падеж или время. Это отдельные аннотации текста, а не готовое решение прикладной задачи. Ошибка в выборе lemma может объединить разные слова или разделить употребления одного слова, изменив статистику features.

Контекст и инструмент

Некоторые инструменты перечисляют возможные разборы по словарю, другие выбирают вариант с учётом контекста. Поэтому наличие морфологического словаря и наличие contextual model — разные возможности. Для text classification morphological analysis может быть полезен, но не является обязательным: character n-grams иногда позволяют работать со словоформами без отдельного разбора.

Stop words и feature selection

Stop words

Stop words — заранее выбранные элементы, которые исключаются из representation. Часто это служебные слова, широко представленные в corpus. Универсального списка для всех задач нет: предлоги, местоимения и отрицания могут нести полезную информацию. Например, удаление «не» меняет отличие между «рекомендую» и «не рекомендую».

Feature selection по данным

Параметры min_df и max_df ограничивают vocabulary по document frequency. Они решают другую задачу, чем лингвистический список stop words. Редкий элемент может быть шумом или важным сигналом класса; частый — общим фоном или частью устойчивой конструкции. Параметры выбирают по dev или внутри cross-validation, сохраняя test для итоговой оценки.

Negation и stop-word removal

Удаление частого служебного слова может сделать противоположные оценки неразличимыми для выбранного представления.

Исходный текст	После удаления «не»	Смысловой эффект
Товар понравился	Товар понравился	Положительная оценка
Товар не понравился	Товар понравился	Отрицание исчезло
Не могу рекомендовать	Могу рекомендовать	Смысл изменился
Совсем не плохой товар	Совсем плохой товар	Оценка искажена

Список stop words следует оценивать вместе с задачей и представлением текста. Частота элемента не означает, что его можно безопасно удалить.

Потери информации при preprocessing

Сигналы, которые могут исчезнуть

Удаление пунктуации объединяет вопрос и утверждение, изменение регистра скрывает отдельные названия, а замена чисел общим маркером убирает конкретные значения. Агрессивная очистка может уничтожить информацию о сумме, дате или количестве. В анализе тональности также имеют значение усилители, отрицания и отдельные особенности эмоциональной записи.

Проверка допустимости преобразований

Для каждой операции полезно рассмотреть пары текстов, которые после обработки становятся одинаковыми. Если эти тексты должны получать разные ответы, преобразование удаляет потенциально полезный сигнал. Окончательное решение принимают по качеству задачи и разбору ошибок. Единого универсального набора очистки для классификации, поиска и извлечения фактов не существует.

Выбор preprocessing для задачи

Каждую операцию связывают с ожидаемой пользой и возможной потерей информации. Один и тот же шаг может помогать одной задаче и мешать другой.

Исходный текст	Методический комментарий
«НЕ рекомендую!!!»	Сохранить отрицание, проверить роль регистра и пунктуации
«Заказ №48392 задержали»	Рассмотреть маскирование идентификатора, сохранить событие
«Классный товар :)»	Учесть смайлик как возможный эмоциональный сигнал
« пришло быстро »	Устранить внешние и повторяющиеся пробелы

Операции добавляют последовательно и сравнивают на одинаковом разбиении. Анализ примеров и ошибок помогает связать изменение качества с конкретным преобразованием.

Corpus statistics

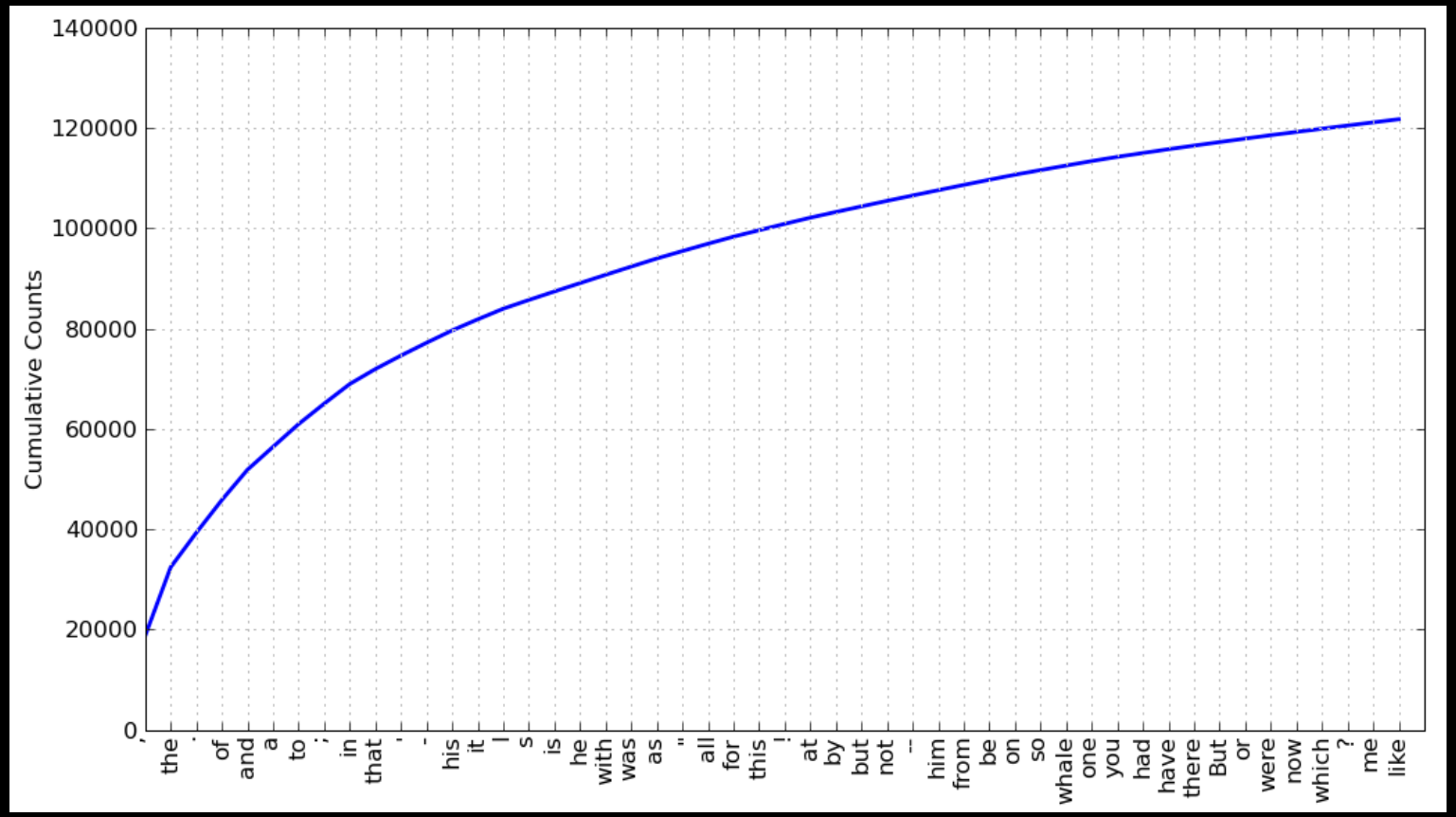
Token frequency и document frequency

Частота токена показывает число его вхождений в выбранном тексте или корпусе. Документная частота показывает число документов, в которых он встретился хотя бы один раз. Например, многократное повторение слова в одном отзыве повышает частоту вхождений, но добавляет только один документ к документной частоте этого слова.

Интерпретация распределений

Частотный анализ помогает обнаруживать характерную лексику, технический шум, дубликаты и особенности источников. Очень частые элементы могут отражать шаблон оформления, а редкие элементы могут быть терминами или опечатками. Полезно сравнивать распределения по классам и источникам, учитывая объём каждой группы и не выводя смысловую значимость только из частоты.

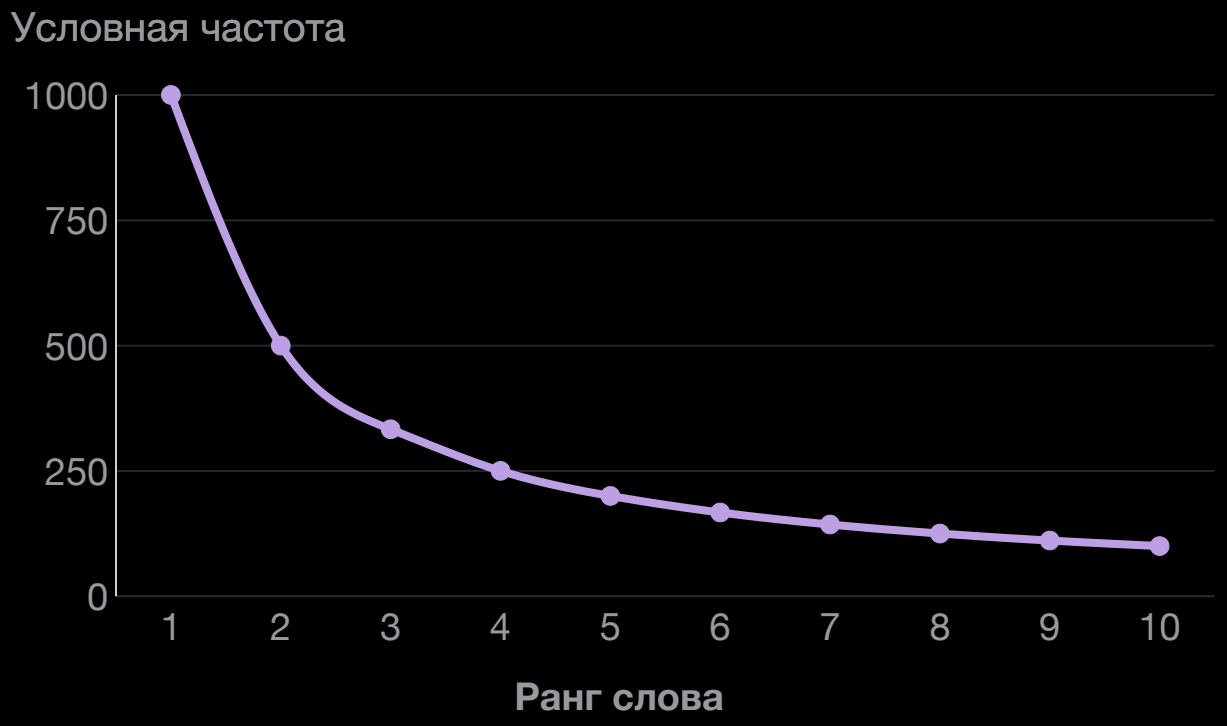
Token frequency



По X: частые токены. По Y: сумма их вхождений. Данные Moby Dick из NLTK Book.

Источник: NLTK Book, гл. 1

Zipf’s law



Если упорядочить слова по убыванию частоты, небольшая часть слов окажется очень частой, а многие слова будут редкими. Zipf’s law описывает приближённое убывание частоты с ростом ранга.

Для обработки текста это означает неравномерное распределение наблюдений: распространённые слова дают много примеров, а редкие требуют осторожной оценки. Кривая не определяет полезность слова для конкретной задачи.

График иллюстрирует условную зависимость «частота = 1000 / ранг». Это схема закономерности, а не измерение учебного корпуса.

Word n-grams и character n-grams

Последовательности соседних элементов

N-gram содержит n последовательных элементов. Для фразы «товар не понравился» unigram features — отдельные слова; bigram features — «товар не» и «не понравился». Word n-grams сохраняют локальный порядок. Character n-grams описывают фрагменты написания и могут учитывать части слов, окончания и характерные сочетания символов.

Выбор длины и уровня

Увеличение длины позволяет учитывать более специфичные конструкции, но уменьшает число повторений каждой из них и расширяет пространство признаков. Bigrams могут сохранить связь отрицания с оценкой, однако не описывают весь контекст предложения. Диапазон длин выбирают на валидации, учитывая размер корпуса, изменчивость формулировок и вычислительные затраты.

Примеры n-grams

Уровень анализа задаёт элементы последовательности, а n определяет число соседних элементов в каждом признаке.

Представление	Исходная запись	Полученные элементы
Word unigrams	товар не пришёл	товар · не · пришёл
Word bigrams	товар не пришёл	товар не · не пришёл
Character trigrams	товар	тов · ова · вар
Character 4-grams	товар	това · овар

Для character n-grams отдельно определяют обработку пробелов и границ слов. Схема показывает локальные фрагменты, которые могут стать столбцами матрицы признаков.

N-gram features и language modeling

Markov Assumption

> The probability of a word only depends on a **fixed** number of previous words.

For an n-gram model,

$$P(y_t \mid \underbrace{y_1, y_2, \ldots, y_{t-1}}_{\text{all previous tokens}}) = P(y_t \mid \underbrace{y_{t-n+1}, \ldots, y_{t-1}}_{\text{n-1 previous token}})$$

N-gram language model ограничивает контекст предыдущими n – 1 tokens. N-gram features используются для описания документа.

Bag of Words

Каждому элементу словаря соответствует отдельный столбец. Документ получает числовой вектор, а значения показывают число вхождений выбранных слов.

Текст	товар	хороший	плохой
«товар хороший»	1	1	0
«товар плохой»	1	0	1
«хороший хороший товар»	1	2	0

В этом примере порядок слов не сохраняется. Повторение «хороший» увеличивает счётчик, но само по себе не задаёт силу оценки. Для учёта локального порядка в словарь можно добавить n-grams.

Term–document matrix

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	…
Antony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
…							

► **Figure 1.1** A term-document incidence matrix. Matrix element (t, d) is 1 if the play in column d contains the word in row t, and is 0 otherwise.

В учебном примере строки соответствуют терминам, столбцы соответствуют произведениям. Единица означает присутствие термина, ноль означает его отсутствие. Это бинарное представление: число повторений не учитывается. В библиотеках машинного обучения часто используют транспонированную ориентацию, где документам соответствуют строки, а признакам соответствуют столбцы.

Матрица из Introduction to Information Retrieval. Ориентацию осей необходимо проверять перед интерпретацией.

Источник: Manning, Raghavan, Schütze, Introduction to IR

Sparse representation

Условная матрица содержит четыре документа и пять выбранных слов. Значения показывают число вхождений.

Документ	товар	доставка	цена	экран	батарея
A	1	0	0	0	0
B	1	1	0	0	0
C	0	0	1	1	0
D	0	0	0	0	2

Из 20 ячеек только 6 ненулевые. Разреженный формат хранит ненулевые значения и их позиции, что особенно полезно при большом словаре.

TF-IDF

Term frequency и inverse document frequency

TF описывает, как часто term встречается в данном документе. IDF зависит от document frequency: сколько документов corpus содержат этот term. TF-IDF объединяет локальную частоту с распространённостью по коллекции. Координата получает больший вес, когда элемент характерен для документа и относительно редко встречается в других документах.

Интерпретация веса

TF-IDF — способ построить features, а не оценка смысла, истинности или тональности. Высокий вес может получить полезное слово, редкое имя или случайная опечатка. Конкретное значение зависит от определения TF, smoothing, log scaling и normalization. Vocabulary и IDF вычисляют на train; при обработке dev, test и новых текстов используют уже полученные параметры.

TF-IDF: пример на небольшой коллекции

Учебная коллекция

Рассмотрим три документа: «доставка быстрая», «доставка дорогая» и «аккумулятор хороший». Слово «доставка» встречается в двух документах, а «аккумулятор» встречается в одном. Если частота каждого рассматриваемого слова внутри своего документа равна единице, различие их ненормированных весов определяется компонентой IDF и выбранным сглаживанием.

Что показывает сравнение

При стандартном убывании IDF с документной частотой более редкое слово «аккумулятор» получает больший коэффициент IDF. Это помогает отличить третий документ от остальных по лексике. Из примера нельзя заключить, что аккумулятор важнее доставки для пользователя. После нормировки сравнение отдельных чисел также зависит от остальных признаков каждого документа.

Document frequency и IDF

Условный корпус: «товар хороший», «товар плохой», «доставка быстрая». В нём три документа. Ниже показаны четыре выбранных термина.

Термин	Документов с термином	Сглаженный IDF
товар	2	1,288
хороший	1	1,693
плохой	1	1,693
доставка	1	1,693

Расчёт: IDF = 1 + ln((1 + N) / (1 + df)). Это коэффициент IDF до умножения на TF и нормировки вектора. Значения округлены до трёх знаков.

Fit и transform

Оценка статистики по корпусу

При обучении векторизатора определяются словарь и необходимые статистики, например документные частоты. Для классификатора обучение означает подбор параметров связи между признаками и метками. Эти операции используют тренировочные данные. Их результат необходимо сохранить, чтобы новые документы попадали в то же пространство признаков и оценивались согласованно.

Обработка новых объектов

При применении готового преобразования проверочные тексты получают признаки в соответствии с сохранённым словарём. Для словарной модели ранее неизвестные элементы обычно не добавляют новые столбцы автоматически. Изменение словаря требует согласования с классификатором. Поэтому обучение преобразований и получение ответа для новых объектов рассматривают как разные этапы.

Ограничения lexical features

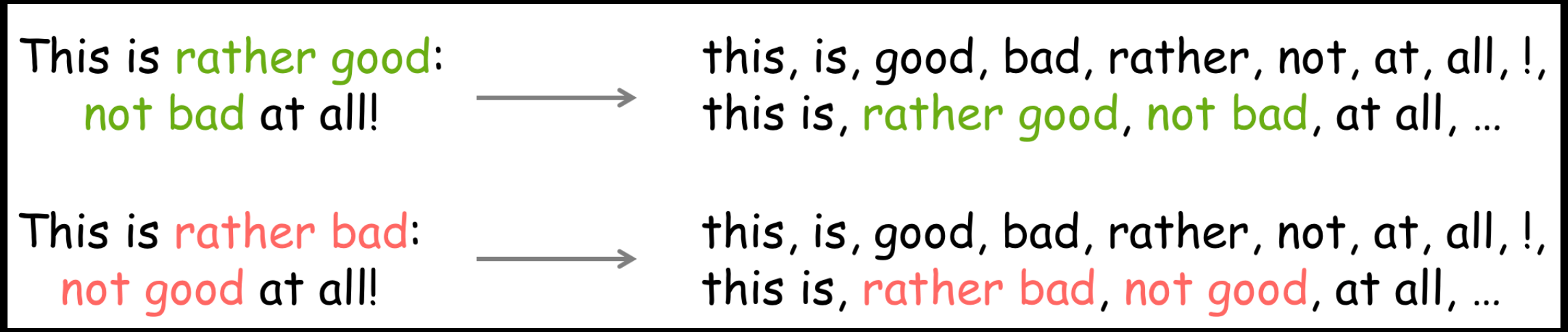
Смысловая близость и совпадение слов

Фразы «купить автомобиль» и «приобрести машину» близки по смыслу, но не имеют общих word unigrams. Лексические признаки непосредственно отражают присутствие элементов словаря. Их успешность зависит от того, насколько устойчиво нужные различия выражаются словами и локальными сочетаниями в доступном корпусе.

Контекст и перенос

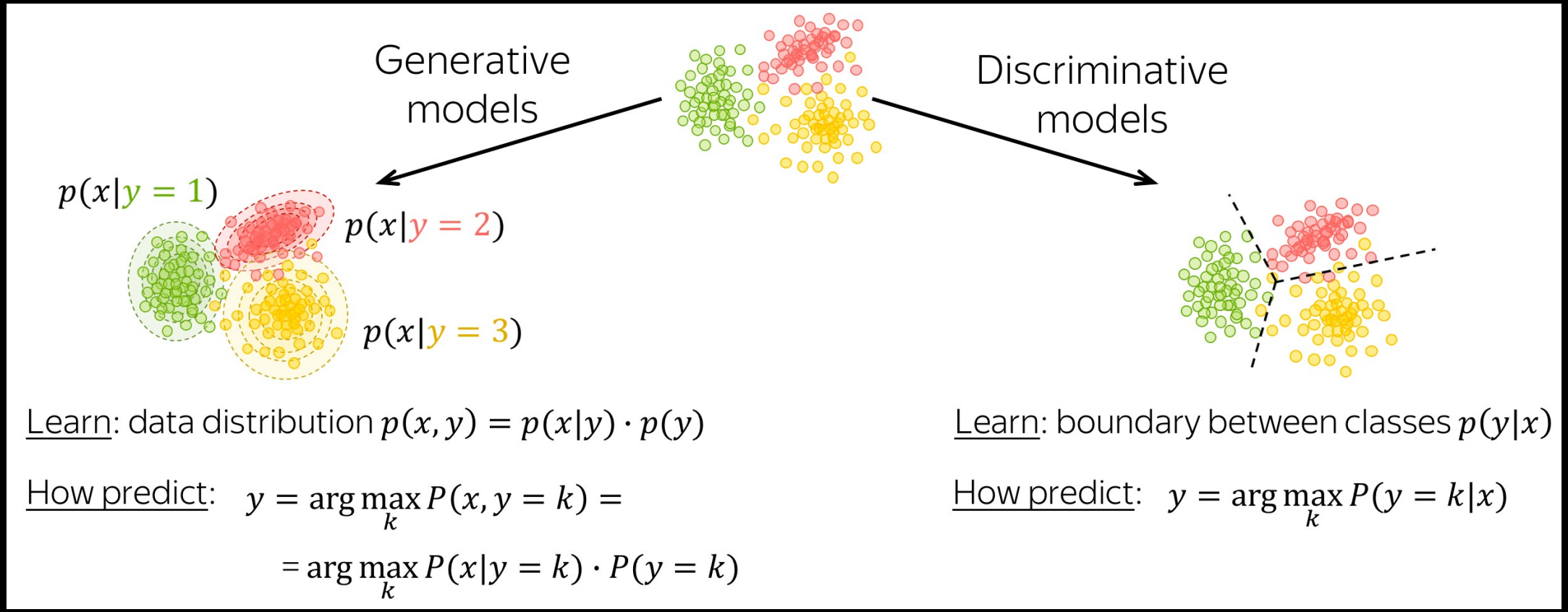
Одинаковые слова могут входить в разные синтаксические конструкции или использоваться с иронией. N-grams учитывают часть локального порядка, но число возможных сочетаний быстро растёт. При изменении тематики появляется новая лексика. Эти ограничения объясняют интерес к другим представлениям текста, при этом TF-IDF остаётся полезной основой для измеримого сравнения.

Feature engineering: n-grams



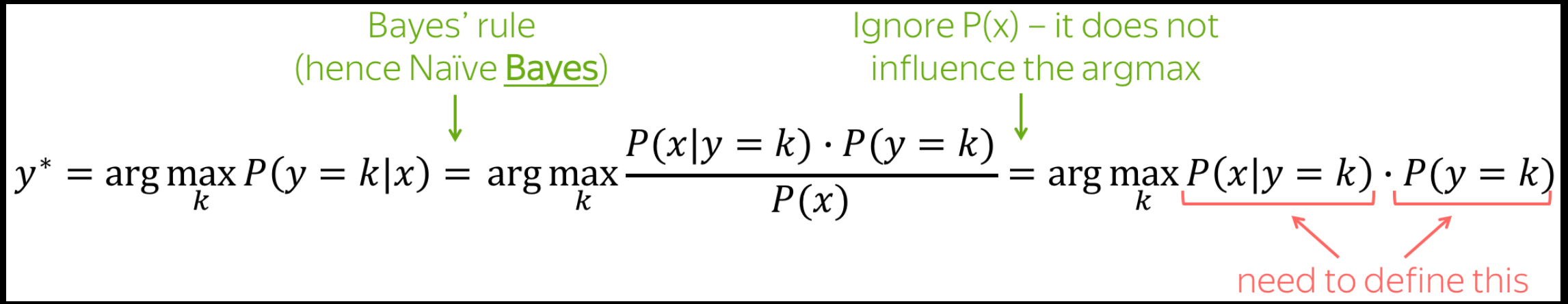
Фразы на рисунке содержат одинаковые unigram features, но выражают противоположные оценки. Word n-grams позволяют отдельно представить сочетания «rather good» и «not bad», отличив их от «rather bad» и «not good». Это частично сохраняет локальный порядок. Vocabulary растёт, поэтому n-gram range и ограничения document frequency выбирают по dev; добавление новых features не гарантирует улучшения для каждого corpus.

Generative и discriminative models



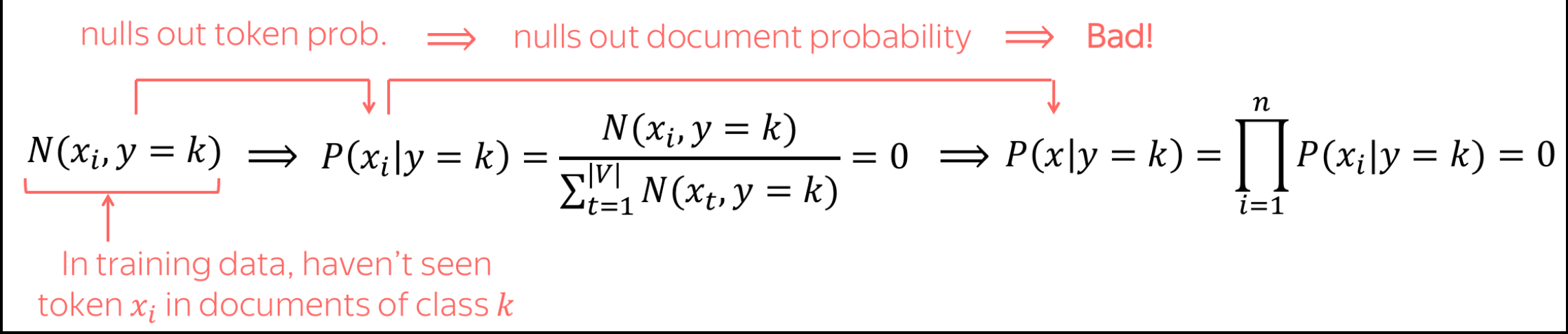
Generative model описывает p(x, y). Discriminative model оценивает p(y | x) или строит decision boundary.

Naive Bayes



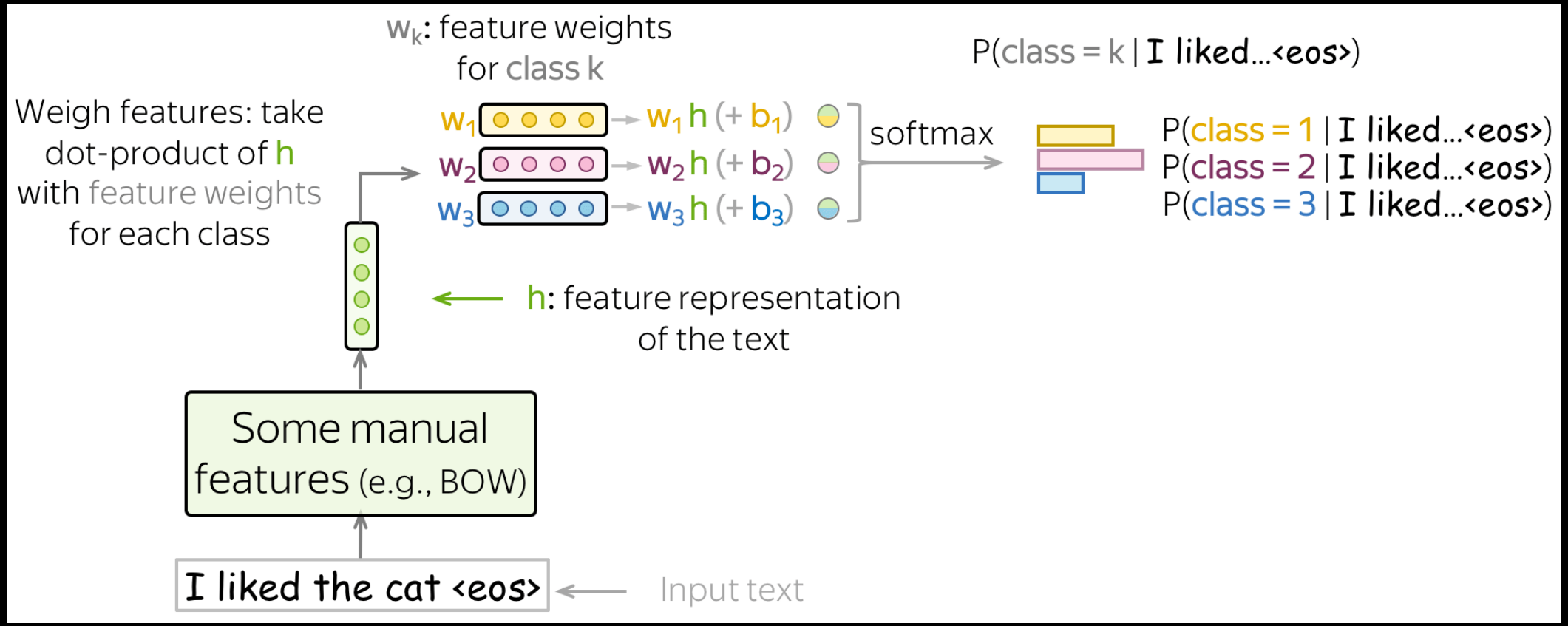
Naive Bayes выбирает класс, который лучше объясняет наблюдаемые features с учётом prior. После появления текста оценивается posterior. Ключевое допущение — conditional independence features при фиксированном классе. Для Multinomial Naive Bayes используются counts и class-specific частоты. Это простое приближение: слова в языке зависимы, поэтому модель удобна как baseline, но не описывает всю структуру высказывания.

Naive Bayes: smoothing



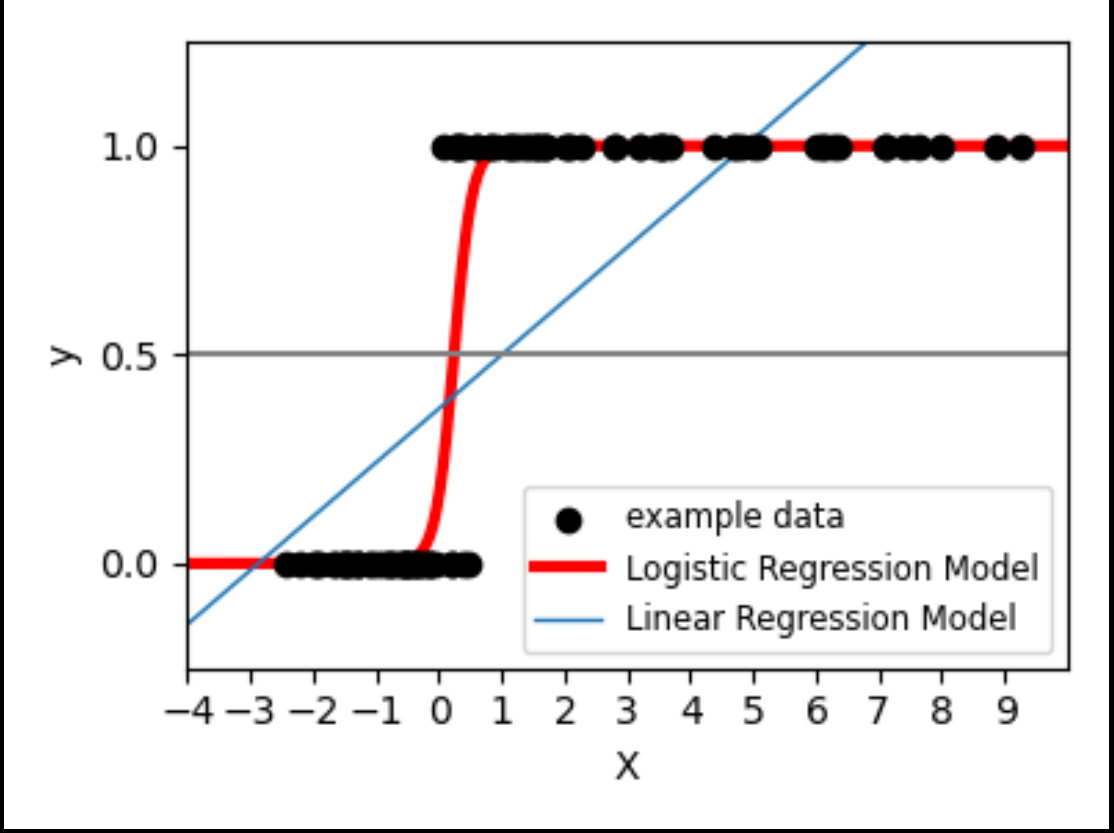
Если слово из vocabulary ни разу не встречалось в документах одного класса, несглаженная оценка его вероятности равна нулю. В произведении такая оценка обнуляет вклад всего документа. Additive smoothing добавляет небольшую положительную величину к counts до normalization. Это позволяет не считать отсутствие наблюдения доказательством невозможности. В sklearn силу smoothing задаёт alpha; значение выбирают на train/dev.

MaxEnt / Logistic Regression



Features h взвешиваются отдельно для каждого класса. Softmax преобразует scores в class probabilities.

Sigmoid и decision threshold



Красная кривая преобразует значение линейной оценки в число между нулём и единицей. Порог определяет, когда система выбирает положительный класс. Чёрные точки показывают метки искусственных примеров.

График иллюстрирует форму зависимости на одном признаке. В текстовой модели линейная оценка обычно объединяет множество словных или символьных признаков.

Учебный пример scikit-learn 1.6. Вероятностная оценка модели не является гарантией правильности конкретного ответа.

Источник: Документация scikit-learn 1.6, учебный пример

Classical classifiers: сравнение

Classifiers получают одинаковый feature representation, но используют разные предположения и способы обучения.

Метод	Принцип	Практическая роль
Naive Bayes	Частоты и conditional independence	Быстрый count-based baseline
Logistic Regression	Линейные scores и class probabilities	TF-IDF baseline на семинаре
Linear SVM	Decision boundary с margin	Альтернатива для sparse features

Linear SVM используют как альтернативу другим линейным classifiers для sparse features. Сравнение моделей проводят на одном split и с одинаковыми metrics.

Text classification pipeline

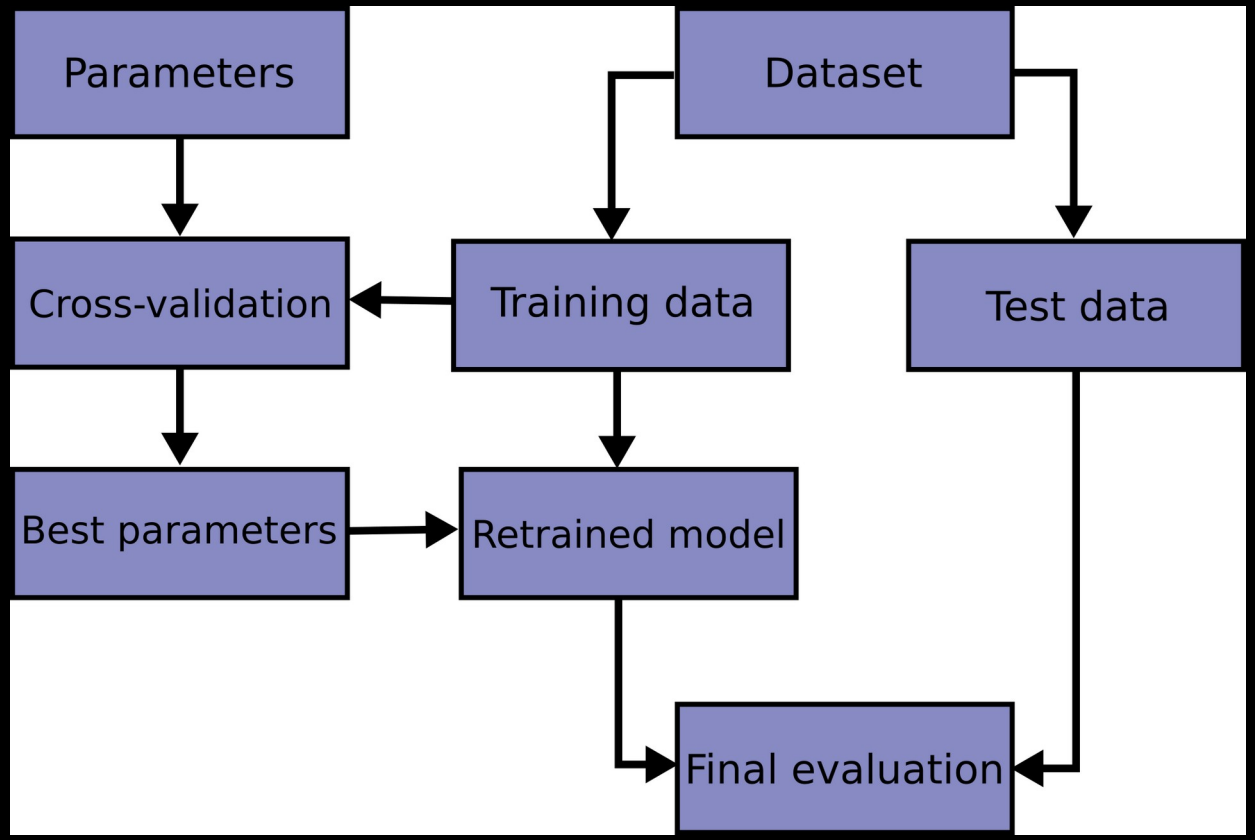
Формирование решения

Для базовой системы сначала определяют метки и разделяют исходные документы. По тренировочным текстам формируют словарь, оценивают IDF и получают матрицу признаков. Затем классификатор связывает эти признаки с известными метками. Сохранённое решение включает не только веса классификатора, но и правила преобразования текста, словарь и статистики.

Получение предсказания

Новый документ обрабатывают тем же способом и представляют в исходном пространстве признаков. Классификатор вычисляет оценки классов, после чего применяется правило выбора ответа. Изменение словаря или нормализации без согласования с обученной моделью нарушает этот процесс. Ошибки разбирают по этапам, отделяя проблемы данных, признаков и самого алгоритма.

Cross-validation и model selection



Сначала из исходного набора выделяют итоговый тест. На оставшихся данных сравнивают настройки с помощью cross-validation. Затем выбранную конфигурацию обучают на тренировочных данных и оценивают на сохранённом тесте.

Тест не должен управлять выбором настроек. Иначе измеренное качество отражает в том числе подстройку к этой выборке.

Схема из документации scikit-learn. Обучаемую векторизацию включают внутрь каждого разбиения cross-validation.

Источник: Документация scikit-learn

Data leakage

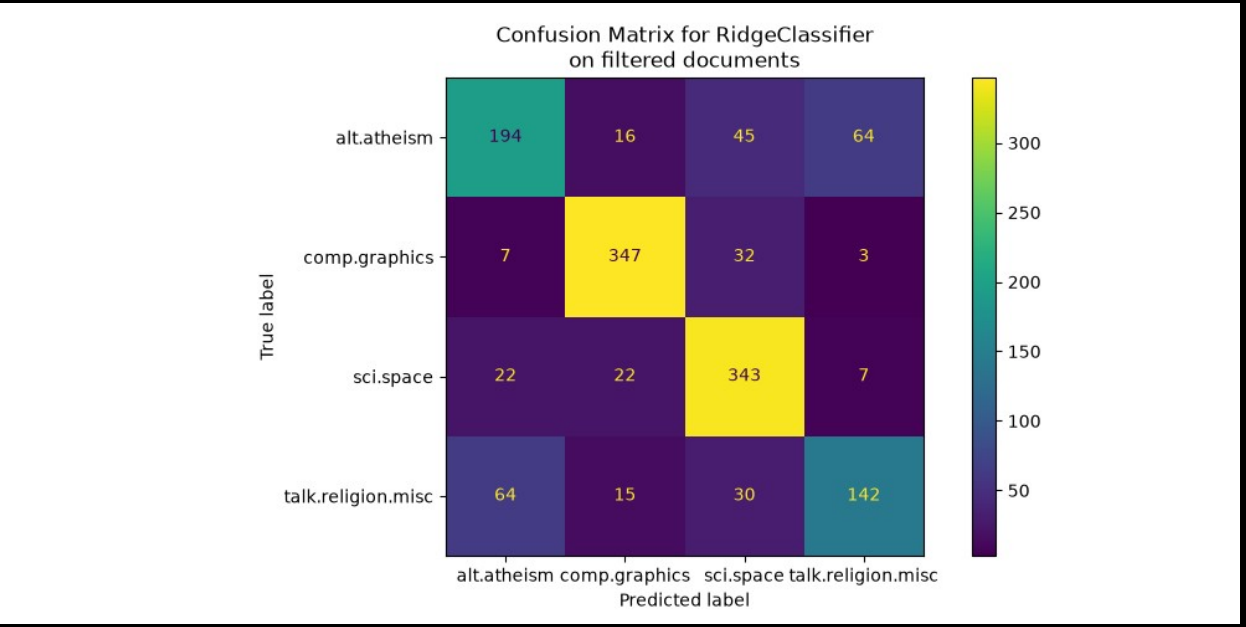
Связь этапов

Конвейер объединяет подготовку признаков и конечную модель в согласованную последовательность. Сначала выделяют тренировочные и проверочные объекты. Затем статистики преобразований и параметры классификатора оценивают только по тренировочной части. Новые тексты проходят те же преобразования с уже сохранёнными настройками и получают предсказания модели.

Границы защиты

При cross-validation все обучаемые преобразования должны заново оцениваться внутри каждой тренировочной части. Это предотвращает использование статистики проверочных объектов. Однако сам конвейер не устраняет дубликаты, связи между авторами или ошибки временного разбиения. Эти источники утечки требуют отдельного контроля на уровне подготовки данных.

Confusion matrix и evaluation



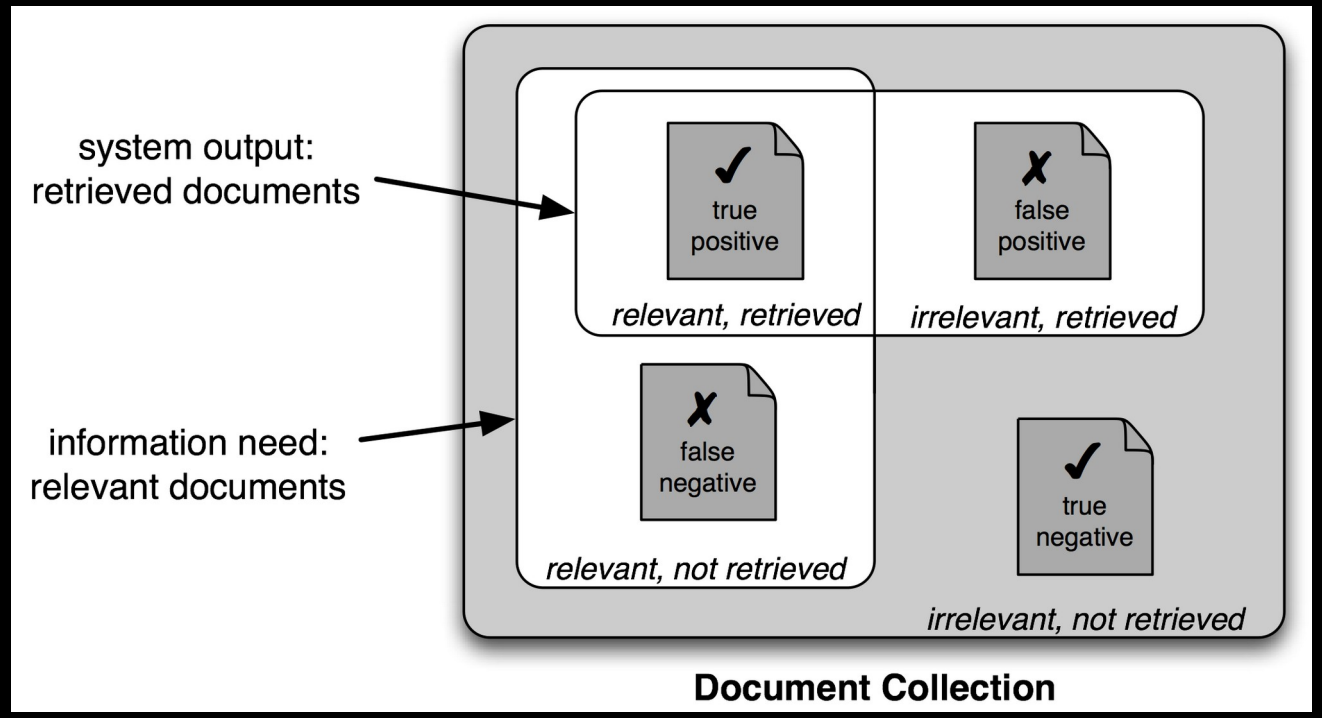
Строки матрицы соответствуют истинным классам, столбцы соответствуют предсказаниям. Диагональ содержит правильные ответы, остальные ячейки показывают ошибки.

Accuracy показывает долю верных меток. Precision измеряет точность ответов класса, recall описывает полноту его обнаружения. F1 является гармоническим средним precision и recall. Macro-F1 усредняет F1 классов с равными весами.

RidgeClassifier, 20 Newsgroups. Результат примера авторов документации, не нашего эксперимента.

Источник: Пример из документации scikit-learn

Precision и recall



True positive соответствует найденному объекту нужного класса. False positive является лишним срабатыванием, а false negative является пропуском нужного объекта. True negative соответствует правильному отклонению.

Precision оценивает долю верных среди выбранных объектов. Recall оценивает, какую долю всех нужных объектов система нашла.

Иллюстрация NLTK Book для поиска. Та же схема применяется к бинарной классификации при заданном положительном классе.

Источник: NLTK Book, гл. 6

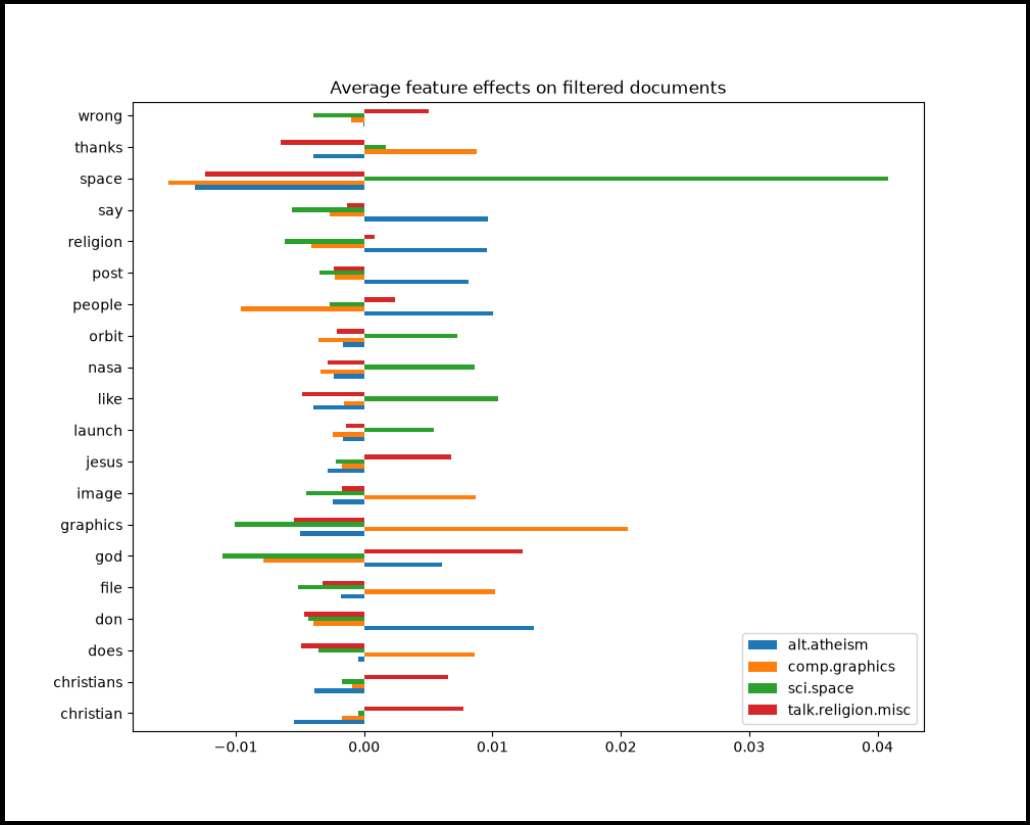
Class imbalance и macro-F1

В условной выборке 90 нейтральных отзывов и 10 жалоб. Система всегда предсказывает нейтральный класс.

Истинный класс	Предсказано: нейтральный	Предсказано: жалоба
Нейтральный	90	0
Жалоба	10	0

Accuracy равна 90%, но recall класса «жалоба» равен 0%. F1 нейтрального класса ≈ 0,947, F1 класса «жалоба» = 0; macro-F1 ≈ 0,474. Равный вес классов в macro-F1 позволяет заметить провал на редком классе.

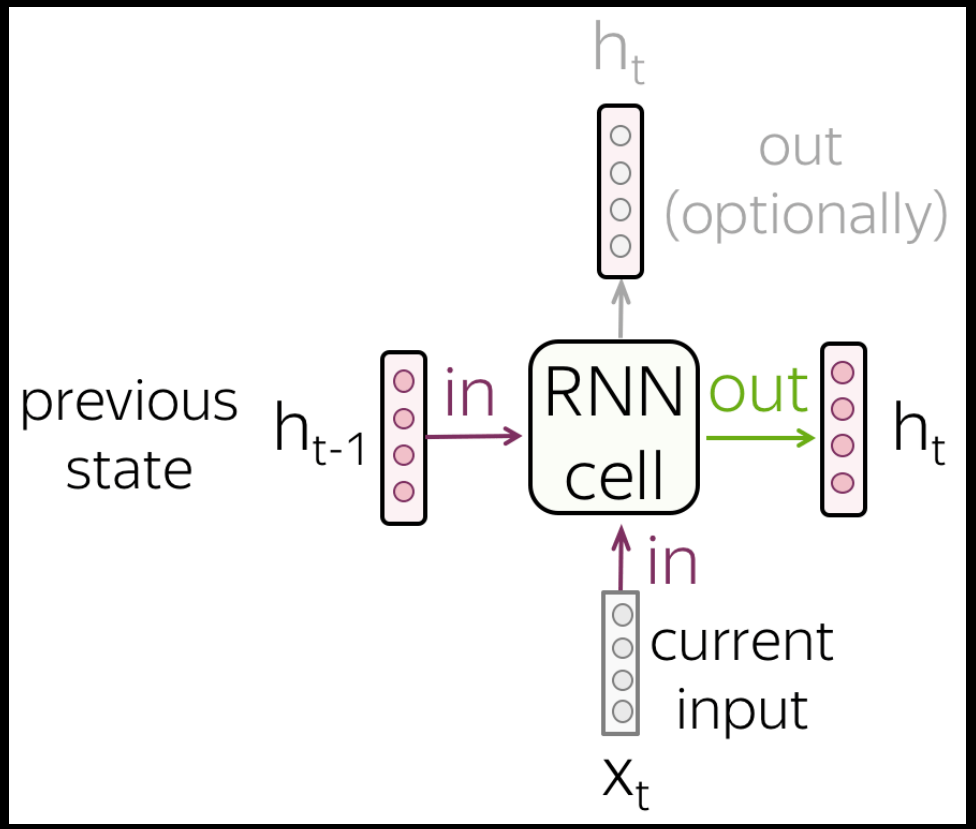
Analysis and interpretability



На рисунке показаны average feature effects: коэффициенты RidgeClassifier умножены на средние значения features на train. Знак показывает направление вклада в score класса. Например, слово space связано с темой sci.space. Это результат примера scikit-learn на 20 Newsgroups. Большой вклад не доказывает причинную связь: модель может использовать шаблоны и особенности dataset. Поэтому интерпретацию дополняют анализом ошибочных примеров.

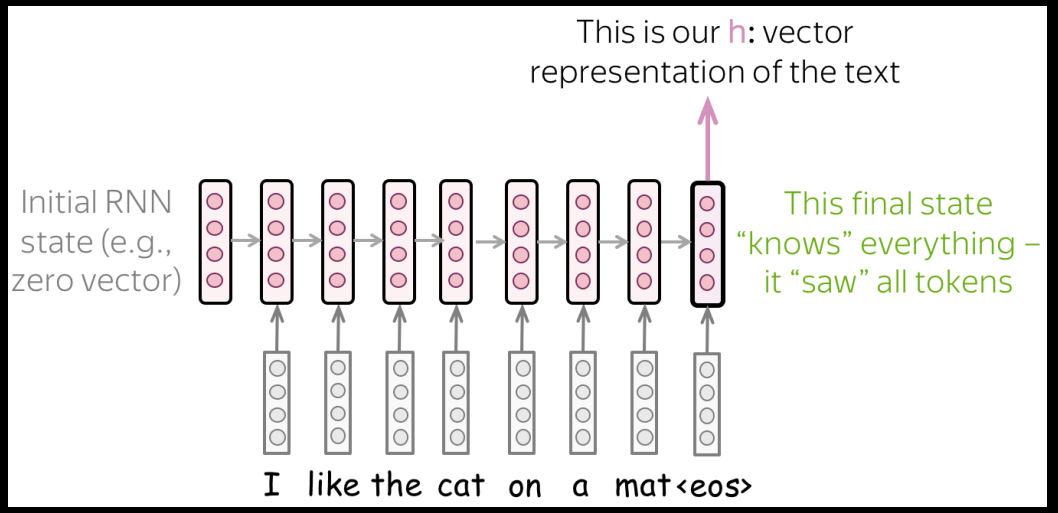
Источник: Документация scikit-learn, 20 Newsgroups

Recurrent Neural Networks



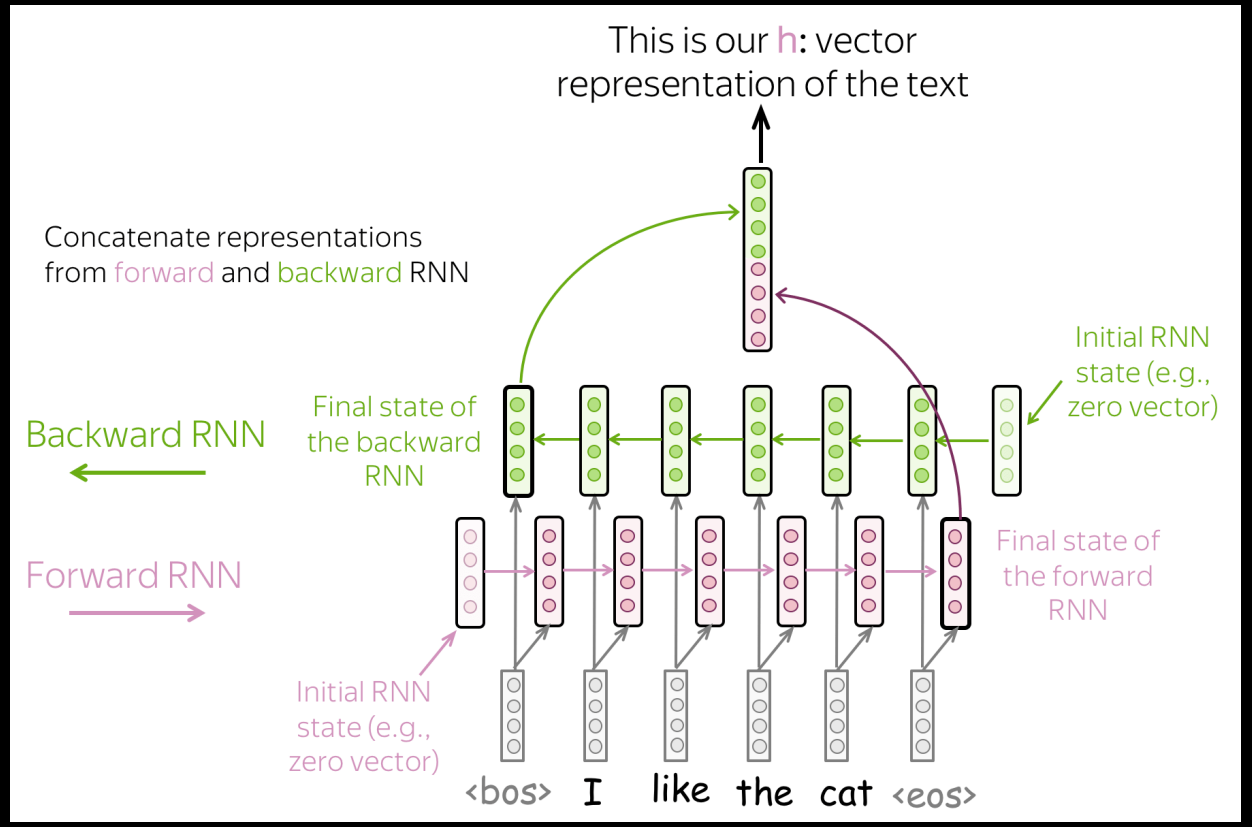
Recurrent Neural Network обрабатывает последовательность по шагам. На каждом шаге получает текущий input x_t и предыдущий hidden state h_{t-1}, после чего вычисляет новое h_t. Hidden state сохраняет информацию, доступную для следующих шагов. Одна и та же операция и её параметры применяются к разным позициям. Такая организация учитывает порядок tokens и позволяет обрабатывать последовательности разной длины.

RNN для text classification



Для text classification каждому token соответствует embedding. RNN читает последовательность; её последний hidden state можно использовать как representation всего текста. Затем classifier предсказывает class label. На схеме состояния связаны во времени, а параметры перехода общие для всех шагов. При работе с batch необходимо учитывать реальные длины: padding не должен случайно подменять содержательный последний state.

Bidirectional RNN



Bidirectional RNN использует два направления чтения: от начала к концу и от конца к началу. Их representations объединяются, чтобы учитывать контекст с обеих сторон. Это применимо, когда весь входной текст уже доступен, например при classification или sequence labeling. При autoregressive generation будущие tokens неизвестны, поэтому обратное чтение целевой последовательности использовать нельзя.

RNN: long-range dependencies

Long-range dependencies

Информация о ранних tokens должна пройти через последовательность hidden states. При обучении vanilla RNN сигнал может ослабевать или нестабильно возрастать; это связано с vanishing и exploding gradients. В результате связь между удалёнными частями текста бывает трудно выучить. Само наличие hidden state не гарантирует, что модель сохраняет все важные детали.

Последовательность вычислений

В стандартной RNN следующий state зависит от предыдущего, поэтому позиции внутри последовательности нельзя обработать полностью независимо. Это ограничивает parallelization и усложняет работу с длинным контекстом. В classification с последним state информация о документе дополнительно проходит через представление фиксированного размера. Ограничения мотивировали gated RNN и механизмы attention.

LSTM и GRU

Gated RNN

LSTM и GRU — варианты RNN, в которых gates регулируют обновление state и сохранение информации. LSTM дополнительно выделяет cell state; GRU использует другую, более компактную схему обновления. На этом этапе достаточно понимать назначение gates: модель учится, какую информацию продолжать хранить, а какую заменять новым входом.

Назначение и ограничения

Такие модели применялись в text classification, sequence labeling и language modeling. Gates облегчают обучение зависимостей на расстоянии, но не устраняют последовательный характер обработки и не обеспечивают неограниченную память. Выбор конкретной cell и её формулы не входят в задачи первого занятия; важно отличать семейство RNN от конкретных вариантов LSTM и GRU.

RNN, attention и Transformer

Attention

Attention позволяет выходу использовать разные части доступного контекста с разными весами. В encoder–decoder системах он уменьшает необходимость передавать всё содержание через единственный вектор. Механизм может применяться вместе с RNN; поэтому attention и Transformer не являются взаимозаменяемыми названиями одного понятия.

Transformer и дальнейшие темы

Transformer строит взаимодействие позиций через attention и использует информацию об их порядке. В отличие от стандартной recurrent схемы, обработку доступной последовательности при обучении можно сильнее parallelize. Подробные механизмы attention, masking и варианты encoder/decoder относятся ко второму блоку. На первой лекции важна связь этого этапа с ограничениями RNN.

Библиотеки для NLP

Инструменты preprocessing и classical ML

NLTK предоставляет учебные corpora и алгоритмы обработки текста. spaCy организует linguistic annotation через pipeline и объекты Doc, Token, Span. scikit-learn предоставляет vectorizers, classifiers, Pipeline и evaluation. Их функции пересекаются частично: библиотека выбирается по нужной операции, языку и способу построения эксперимента.

Pretrained representations и models

Gensim используется для corpus transformations и word embeddings. Hugging Face объединяет модели, datasets и средства tokenization; Sentence Transformers ориентирован на representations целых текстов. Программный пакет, pretrained weights и dataset — разные компоненты. Для воспроизводимости сохраняют их версии, настройки обработки и правила разбиения данных.

NLTK

Анализ языка и корпусов

NLTK объединяет алгоритмы tokenization, stemming, отдельные средства lemmatization и доступ к корпусам. Частотные распределения помогают описывать лексику, а инструменты n-grams и коллокаций позволяют изучать совместную встречаемость элементов. Такая организация удобна для поэтапного разбора операций и сравнения нескольких вариантов обработки.

Выбор отдельных инструментов

NLTK не задаёт единственную обязательную последовательность анализа. Пользователь выбирает алгоритмы и определяет, как передавать результаты между ними. Поддержку языка проверяют отдельно для каждого метода и ресурса. Наличие функции с нужным названием не означает, что она пригодна для русского текста или конкретного жанра без дополнительных условий.

NLTK: corpora и lexical resources

Состав ресурсов

Корпуса содержат тексты и иногда аннотации, словари описывают лексические единицы, а списки stop words задают набор часто исключаемых элементов. Эти ресурсы имеют собственное происхождение и назначение. Многие из них доступны отдельно от программного пакета, поэтому необходимо понимать, какие именно данные использует выбранная операция.

Применимость к задаче

Готовый ресурс отражает определённый язык, предметную область и правила составления. Например, список stop words может содержать отрицания, которые важны для оценки тональности. Перед применением проверяют состав ресурса на характерных текстах. Для сравнения методов сохраняют одну и ту же версию, чтобы изменение результата можно было связать с конкретным решением.

spaCy: Doc, Token и Span

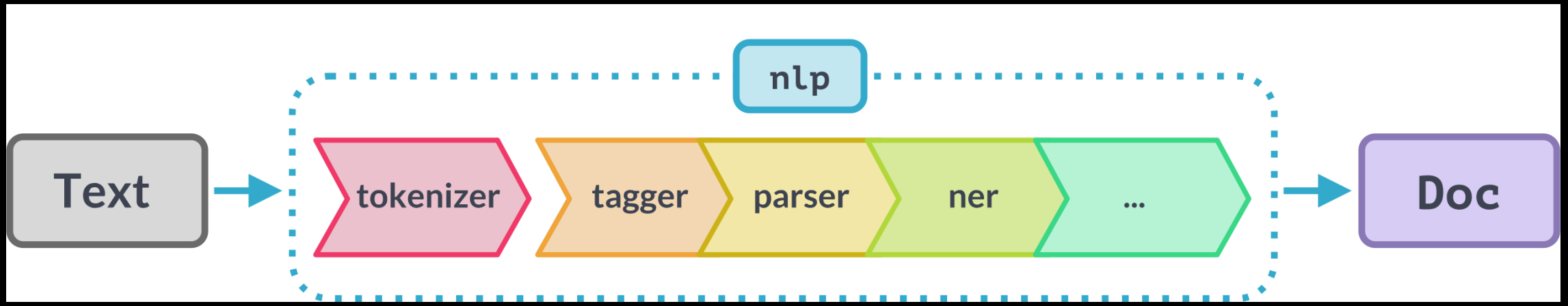
Документ и токен

spaCy представляет обработанный текст объектом документа, который сохраняет исходное содержание и последовательность токенов. Каждый токен связан с позицией в тексте и доступными аннотациями: леммой, частью речи и другими признаками. Набор заполненных атрибутов зависит от компонентов, которые обработали документ, и возможностей загруженной модели.

Фрагменты текста

Фрагмент объединяет последовательные токены и позволяет описывать предложение, именованную сущность или иной выделенный участок. Такое представление связывает лингвистические аннотации с исходной записью. Для последующего анализа важно сохранять границы фрагментов и не путать предсказание компонента с достоверной ручной разметкой.

spaCy pipeline



Состав компонентов зависит от выбранной языковой модели.

Tokenizer выделяет элементы текста. Последующие компоненты добавляют части речи, синтаксические связи, сущности и другие аннотации к документу. Схема показывает типовой принцип, а конкретный состав зависит от модели. Некоторые этапы используют результаты предыдущих. Поэтому отключение компонента требует проверки оставшихся аннотаций и качества конечной задачи.

Источник: Документация spaCy

spaCy: language data и trained pipelines

Базовая конфигурация языка

Языковой объект может содержать tokenizer, правила и общие настройки без обученных компонентов анализа. Это позволяет разделить техническое выделение элементов и предсказание лингвистических характеристик. Наличие поддержки языка в библиотеке само по себе не означает, что система уже умеет определять части речи, сущности и леммы с заданным качеством.

Обученные компоненты

Предобученная модель поставляет параметры и конфигурацию для конкретного набора операций. Её возможности зависят от данных обучения, языка и состава пакета. При выборе проверяют назначение модели и примеры её ошибок на целевом корпусе. Размер или название варианта помогают идентифицировать пакет, но не заменяют измерение качества в конкретном применении.

scikit-learn: vectorizers и Pipeline

CountVectorizer и TfidfVectorizer

CountVectorizer формирует vocabulary и counts выбранных features. TfidfVectorizer дополнительно рассчитывает IDF и применяет TF-IDF weighting. Настройки включают word или character analyzer, n-gram range, min_df и max_df. Vectorizer обучается на train; готовый vocabulary применяется к dev, test и новым текстам без переобучения.

Pipeline и classifier

Pipeline объединяет vectorizer с LogisticRegression, MultinomialNB или другим estimator. В cross-validation каждый train fold получает собственный fitted vectorizer, что снижает риск data leakage. Методы fit, transform и predict относятся к разным операциям. На семинаре эта схема станет основой baseline; здесь рассматриваются назначение компонентов и правила их использования.

Gensim и word embeddings

Corpus transformations

Gensim предоставляет Dictionary для vocabulary, BoW representations и transformations, включая TF-IDF и LSI. Инструменты поддерживают обработку коллекций по документам, что полезно, когда corpus не помещается в память целиком. Входные и выходные representations должны сохранять согласованные идентификаторы features; одинаковая размерность ещё не гарантирует их соответствие.

Word embeddings

В Gensim есть реализации Word2Vec и FastText, а KeyedVectors используется для хранения и применения готовых vectors. Word embeddings описывают слова; получение representation всего документа требует отдельного решения. Word2Vec, GloVe, fastText и doc2vec относятся ко второй лекции. Gensim позволяет сохранять representations и применять их при обработке нового corpus.

Hugging Face ecosystem

Модели и подготовка данных

Transformers предоставляет интерфейсы к архитектурам, предобученным параметрам и связанным tokenizers. Datasets помогает загружать и преобразовывать наборы данных. Tokenizers содержит средства tokenization. Совместимость tokenizer с моделью имеет принципиальное значение, поскольку словарь и правила выделения элементов определяют представление входного текста.

Выбор готового компонента

В Hub размещаются модели и наборы данных с описаниями назначения и ограничений. При выборе проверяют поддерживаемые языки, целевую задачу, входной формат и условия использования. Для повторения результата фиксируют конкретную ревизию. Наличие готового компонента сокращает объём разработки, но его качество на целевых текстах требует отдельной проверки.

Sentence Transformers

Представления целых текстов

sentence-transformers предоставляет модели и средства получения векторов предложений и документов. Такие представления используют для семантического поиска, кластеризации и как признаки для других алгоритмов. В типичном сценарии каждому тексту соответствует один вектор, а смысловая близость оценивается выбранной мерой в пространстве представлений.

Условия применения

Необходимо проверить язык модели, назначение её обучения и требования к оформлению входа. Для поиска некоторые модели различают запросы и документы или используют специальные инструкции. Полученный вектор сам по себе не является меткой тональности. При сравнении с TF-IDF сохраняют одинаковые данные и процедуру оценки, чтобы измерить влияние способа представления.

Контрольные вопросы

Данные и representations

Чем отличаются single-label и multi-label classification? Как различаются token, type и vocabulary? Когда lemmatization может потерять полезную информацию? Почему удаление negation опасно для sentiment analysis? Что сохраняют word n-grams и что теряет Bag of Words? Чем document frequency отличается от числа вхождений слова?

Модели и evaluation

Как разделяются feature extractor и classifier? В чём смысл conditional independence у Naive Bayes? Какова роль Logistic Regression в TF-IDF baseline? Что хранит hidden state RNN и зачем нужны LSTM/GRU? Почему vocabulary оценивают только на train? Как выбрать split и metrics, если классы несбалансированы, а документы связаны между собой?

Результаты лекции и содержание семинара

Основные понятия

NLP-задача определяется входом, доступным контекстом и ожидаемым ответом. Данные и annotation задают смысл меток. Preprocessing и feature representation определяют, какую информацию получит classifier. Classical models служат понятным baseline; RNN показывает, как обучаемое представление может учитывать последовательность tokens. Качество проверяют на независимых данных.

Вторая лекция и семинар

Вторая лекция: word2vec, GloVe, fastText, doc2vec; static и contextual embeddings, ELMo → BERT, transfer learning и роль embeddings в retrieval/RAG. Семинар: preprocessing с NLTK и spaCy, TF-IDF + Logistic Regression через sklearn и сравнение с Sentence Transformers. Все фрагменты кода и выполнение экспериментов относятся к семинару.

Материалы и источники

YSDA NLP course

Репозиторий, lecture slides и notebooks.

NLTK Book и spaCy

Preprocessing и linguistic annotation.

scikit-learn

Vectorizers, Pipeline, classifiers и evaluation.

Gensim · Hugging Face · SBERT

Corpus transformations и pretrained representations.